

Solving a class of multi-scale elliptic PDEs by Fourier-based mixed physics informed neural networks

Xi'an Li^a, Jinran Wu^{b,*}, Xin Tai^a, Jianhua Xu^c, You-Gan Wang^d

^a Ceyear Technologies Co., Ltd, Qingdao 266555, China

^b Australian Catholic University, Brisbane 4000, Australia

^c The 41st Institute of China Electronics Technology Group Corporation, Qingdao, 266555, China

^d The University of Queensland, St Lucia 4067, Australia

ARTICLE INFO

Keywords:

Multi-scale
Rough coefficient
FMPINN
Fourier feature mapping
Flux variable
Reduce order

ABSTRACT

Deep neural networks have garnered widespread attention due to their simplicity and flexibility in the fields of engineering and scientific calculation. In this study, we probe into solving a class of elliptic partial differential equations (PDEs) with multiple scales by utilizing Fourier-based mixed physics informed neural networks (dubbed FMPINN), its solver is configured as a multi-scale deep neural network. In contrast to the classical PINN method, a dual (flux) variable about the rough coefficient of PDEs is introduced to avoid the ill-condition of neural tangent kernel matrix caused by the oscillating coefficient of multi-scale PDEs. Therefore, apart from the physical conservation laws, the discrepancy between the auxiliary variables and the gradients of multi-scale coefficients is incorporated into the cost function, obtaining a satisfactory solution of PDEs by minimizing the defined loss through some optimization methods. Additionally, a trigonometric activation function is introduced for FMPINN, which is suited for representing the derivatives of complex target functions. Handling the input data by Fourier feature mapping will effectively improve the capacity of deep neural networks to solve high-frequency problems. Finally, to validate the efficiency and robustness of the proposed FMPINN algorithm, we present several numerical examples of multi-scale problems in various dimensional Euclidean spaces. These examples cover low-frequency and high-frequency oscillation cases, demonstrating the effectiveness of our approach. All code and data accompanying this manuscript will be publicly available at <https://github.com/Blue-Giant/FMPINN>.

1. Introduction

Multi-scale problems, governed by partial differential equations (PDEs) with multiple scales, are prevalent in diverse scientific and engineering fields like reservoir simulation, high-frequency scattering, and turbulence modeling. This paper focuses on solving the following type of multi-scale problem:

* Corresponding author.

E-mail addresses: lixian9131@163.com (X. Li), ryan.wu@acu.edu.au (J. Wu), taixin@ceyear.com (X. Tai), xujianhua@ei41.com (J. Xu), ygwanguq2012@gmail.com (Y.-G. Wang).

<https://doi.org/10.1016/j.jcp.2024.113012>

Received 28 July 2023; Received in revised form 5 February 2024; Accepted 12 April 2024

Available online 17 April 2024

0021-9991/© 2024 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

$$\begin{cases} -\operatorname{div}\left(A^\varepsilon(\mathbf{x})\nabla u^\varepsilon(\mathbf{x})\right) = f(\mathbf{x}), & \mathbf{x} \in \Omega, \\ B u^\varepsilon(\mathbf{x}) = g(\mathbf{x}), & \mathbf{x} \in \partial\Omega, \end{cases} \quad (1)$$

where Ω is a bounded subset of $\mathbb{R}^d$ ($d = 1, 2, 3, \dots$) with piecewise Lipschitz boundary and satisfies the interior cone condition, ε is a small positive parameter that signifies explicitly the multiscale nature of the rough coefficient $A^\varepsilon(\mathbf{x})$. B is a boundary operator in $\partial\Omega$ that imposes the boundary condition of u^ε , such as Dirichlet, Neumann and Robin. ∇ and div are the gradient and divergence operators, respectively. $f(\mathbf{x}) \in L^2(\Omega)$ is a given function. In addition, $A^\varepsilon(\mathbf{x})$ is symmetric and uniformly elliptic on Ω . It means that all eigenvalues of A^ε are uniformly bounded by two strictly positive constants $\lambda_{\min}(A^\varepsilon)$ and $\lambda_{\max}(A^\varepsilon)$. In other word, for all $\mathbf{x} \in \Omega$ and $\xi \in \mathbb{R}^d$, we have

$$\lambda_{\min}(A^\varepsilon)|\xi|^2 \leq \xi^T A^\varepsilon(\mathbf{x})\xi \leq \lambda_{\max}(A^\varepsilon)|\xi|^2. \quad (2)$$

The multi-scale problem (1) frequently arises in the fields of physical simulations and engineering applications, including the study of flow in porous media and the analysis of mechanical properties in composite materials [1–3]. Generally, the analytical solutions of (1) are seldom available, so solving numerically this problem through approximation methods is necessary. Lots of numerical methods focus on efficient, accurate and stable numerical schemes have gained favorable achievement, such as heterogeneous multi-scale methods [2–4], numerical homogenization [5–7], variational multi-scale methods [8,9], multi-scale finite element methods [10–12], flux norm homogenization [13,14], rough polyharmonic splines (RPS) [15], generalized multi-scale finite element methods [16–18], localized orthogonal decomposition [19,20], etc. In contrast to standard numerical methods including FEM and FDM, they alleviate substantially the computational complexity in handling all relevant scales, improve the numerical stabilities and expedite the convergence. However, they still will encounter the curse of complex domain and dimensionality in general.

Deep neural networks (DNN), an efficient meshfree method without the discretization for a given interested domain, have drawn more and more attention from researchers to solve numerically the ordinary and partial differential equations as well as the inverse problems for complex geometrical domain and high-dimensional cases [21–27], due to their extraordinary universal approximation capacity [28]. Among these methods, the physics-informed neural networks (PINN) dating back to the early 1990s again attracted widespread attention of researchers and have made remarkable achievements for approximating the solution of PDEs by embracing the physical laws with neural networks, on account of the rapid development of computer science and technology [24,29]. This method skillfully incorporates the residual of governing equations and the discrepancy of boundary/initial constraints, then formulates a cost function that can be optimized easily via the automatic differentiation in DNN. Many efforts have been made to further enhance the performance of PINN are concluded as two aspects: refining the selection of the residual term and designing the manner of initial/boundary constraints. In terms of the residual term, there are XPINN [30], cPINN [31], two-stage PINN [32] and gPINN [33], and so on. By subtly encoding the I/B constraints into DNN in a hard manner, the PINN can be easy to train with low computational complexity and obtain a high-precision solution of PDEs with complex boundary conditions [34–36]. Motivated by the reduction of order in conventional methods [12], some attempts have been made to solve the high-order PDEs by reframing them as some first-order systems, this will overcome the shortcomings of the computational burden for high-order derivatives in DNN. For example, the deep mixed residual method [27], the local deep learning method [37] and the deep FOSLS method [38,39].

Many studies and experiments have indicated that the general DNN-based algorithms are commonly used to solve a low-frequency problem in varying dimensional space, but will encounter tremendous challenges for high-frequency problems such as multi-scale PDEs (1). The frequency principle (F-principle) [40] or spectral bias [41] of DNN shows that neural networks are typically efficient for fitting objective functions with low-frequency modes but inefficient for high-frequency functions. Then, a series of multi-scale DNN (MscaleDNN) models were proposed to overcome the shortcomings of normal DNN for high-frequency problems by converting high-frequency contents into low-frequency ones via a radial scale technique [42–45]. After that, some corresponding mechanisms were developed to explain this performance of DNN, such as the Neural Tangent Kernel (NTK) [46,47]. Furthermore, many researchers attempted to utilize a Fourier feature mapping consisting of sine and cosine to improve the capacity of MscaleDNN, which will alleviate the pathology of spectral bias and let neural networks capture high frequencies component effectively [45,46,48–51].

Recently, some works [52,53] have shown that general PINN architecture is unable to capture the multi-scale property of the solution due to the effect of rough coefficient in multi-scale PDEs. Leung et al. [52] proposed a Neural homogenization-based PINN (NH-PINN) method to solve (1), it can well overcome the nonconvergence of normal PINN for multi-scale problems. However, NH-PINN also will encounter the dilemma of dimensional and the burden of computation, because it will convert one low-dimensional problem into a high-dimensional case. By carefully analyzing the NTK matrix associated with the PINN, Carney et al. [53] found that the Forbenius norm of the NTK matrix will become unbound as the oscillation factor ε in A^ε tends to zero. It means that the evolution of residual loss term in PINN will become increasingly stiff as $\varepsilon \rightarrow 0$, then lead to poor training behavior for PINN.

In this paper, a Fourier-based multi-scale mixed PINN (FMPINN) structure is proposed to solve the multi-scale problems (1) with rough coefficients. This method consists of the general PINN architecture and the aforementioned MscaleDNN model with subnetworks being used to capture different frequency components. To overcome the weakness of the normal PINN that failed to capture the jumping gradient information of the oscillating coefficient when tackling the equation in multi-scale PDEs (1), a (dual) flux variable is introduced to alleviate the adverse effect of the rough coefficient. Meantime, it can also reduce the computational burden of PINN for the second-order derivatives of space variables. In addition, the Fourier feature mapping is used in our model to learn each target frequency efficiently and express the derivatives of multi-frequency functions easily, it will remarkably improve the capacity for our FMPINN model to solve multi-scale problems. In a nutshell, the primary contributions of this paper are summarized as follows:

1. We propose a novel neural networks approach by combining normal PINN and MscaledDNN with subnetworks structure to address multi-scale problems, leveraging the Fourier theorem and the F-principle of DNN.
2. Inspired by the reduced order scheme for high-order PDEs, a dual (flux) variable about the rough coefficient of multi-scale PDEs is introduced to address the gradient leakage about the rough coefficient for PINN.
3. By introducing some numerical experiments, we show that the classical PINN method with MscaledDNN solver is still insufficient in providing accurate solutions for multi-scale equations.
4. We showcase the exceptional performance of FMPINN in solving a class of multi-scale elliptic PDEs with essential boundaries in various dimensional spaces. Our method outperforms existing approaches and demonstrates its superiority in addressing these complex problems.

The remaining parts of our work are organized as follows. In Section 2, we briefly introduce the underlying conceptions and formulations for MscaledDNN and the structure of PINN. Section 3 provides a unified architecture of the FMPINN to solve the elliptic multi-scale problem (1) based on its equivalent reduced order scheme, and gives the option of activation function as well as the error analysis of our proposed method. Section 4 details the FMPINN algorithm for approximating the solution of elliptic PDEs with multiple scales. In Section 5, some scenarios of multi-scale PDEs are performed to evaluate the feasibility and effectiveness of our proposed method. Finally, some conclusions of this paper are made in Section 6.

2. Multi-scale physics informed neural networks

2.1. Multi-scale deep neural networks with ResNet technique

The basic concept and formulation of DNN are described briefly in this section, which helps audiences understand the DNN structure through functional terminology. Mathematically, a DNN defines the following mapping

$$\mathcal{F} : \mathbf{x} \in \mathbb{R}^d \implies \mathbf{y} = \mathcal{F}(\mathbf{x}) \in \mathbb{R}^c, \quad (3)$$

with d and c being the dimensions of input and output, respectively. The DNN functional $\mathcal{F}$ is a nested composition of the following single-layer neural unit:

$$\mathbf{y} = \{y_1, y_2, \dots, y_m\} \quad \text{and} \quad y_l = \sigma \left(\sum_{n=1}^d w_{ln} * x_n + b_l \right), \quad (4)$$

where w_{ln} and b_l are called weight and bias of l_{th} neuron, respectively. $\sigma(\cdot)$ is an element-wise non-linear operator, generally referred to as the activation function. Then, we have the following formulation of DNN:

$$\mathbf{y}^{[\ell]} = \sigma \circ (\mathbf{W}^{[\ell]} \mathbf{y}^{[\ell-1]} + \mathbf{b}^{[\ell]}), \quad \text{for } \ell = 1, 2, 3, \dots, L, \quad (5)$$

and $\mathbf{y}^{[0]} = \mathbf{x}$, where $\mathbf{W}^{[\ell]} \in \mathbb{R}^{n_{\ell+1} \times n_{\ell}}$, $\mathbf{b}^{[\ell]} \in \mathbb{R}^{n_{\ell+1}}$ stand for the weight matrix and bias vector of ℓ -th hidden layer, respectively, $n_0 = d$ and n_{L+1} is the dimension of output, and “ $\circ$ ” stands for the elementary-wise operation. For convenience, the output of DNN is denoted by $\mathbf{y}(\mathbf{x}; \theta)$ with θ standing for its all weights and biases.

Residual neural network (ResNet) [54] as a common skillful technique by introducing skip connections between adjacent or nonadjacent hidden layers can overcome effectively the vanishing gradient of parameters in the backpropagation for DNN, then make the network much easier to train and improve well the performance of DNN. Many experiment results showed that the ResNet can also improve the performance of DNN to approximate high-order derivatives and solutions of PDEs [21,27]. We utilize the one-step skip connection scheme of ResNet in this work. Except for the normal data flow, the data will also flow along with the skip connection if the two consecutive layers in DNN have the same number of neurons, otherwise, the data flows directly from one to the next layer. The filtered $\mathbf{y}^{[\ell+1]}(\mathbf{x}; \theta)$ produced by the input $\mathbf{y}^{[\ell]}(\mathbf{x}; \theta)$ is expressed as

$$\mathbf{y}^{[\ell+1]}(\mathbf{x}; \theta) = \mathbf{y}^{[\ell]}(\mathbf{x}; \theta) + \sigma \circ \left(\mathbf{W}^{[\ell+1]} \mathbf{y}^{[\ell]}(\mathbf{x}; \theta) + \mathbf{b}^{[\ell+1]} \right).$$

As we are aware, a normal DNN model is capable of providing a satisfactory solution for general problems. However, it will encounter troublesome difficulty in solving multi-scale problems with high-frequency components. Recently, a MscaledDNN architecture has shown its remarkable performance in dealing with high-frequency problems by converting original data to a low-frequency space [42–44,46]. A schematic diagram of MscaledDNN with Q subnetworks is depicted in Fig. 1.

The detailed procedure of MscaledDNN is described in the following.

1. Generating a scale vector or matrix with Q parts

$$\Lambda = (\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3 \dots, \mathbf{k}_{Q-1}, \mathbf{k}_Q)^T, \quad (6)$$

where $\mathbf{k}_i (i = 1, 2, \dots, Q)$ is a scalar or matrix (trainable or untrainable).

2. Converting the input data $\mathbf{x}$ into $\tilde{\mathbf{x}} = \Lambda \odot \mathbf{x}$ with $\odot$ being the Hadamard product, then feeding $\tilde{\mathbf{x}}$ into the pipeline of MscaledDNN. It is

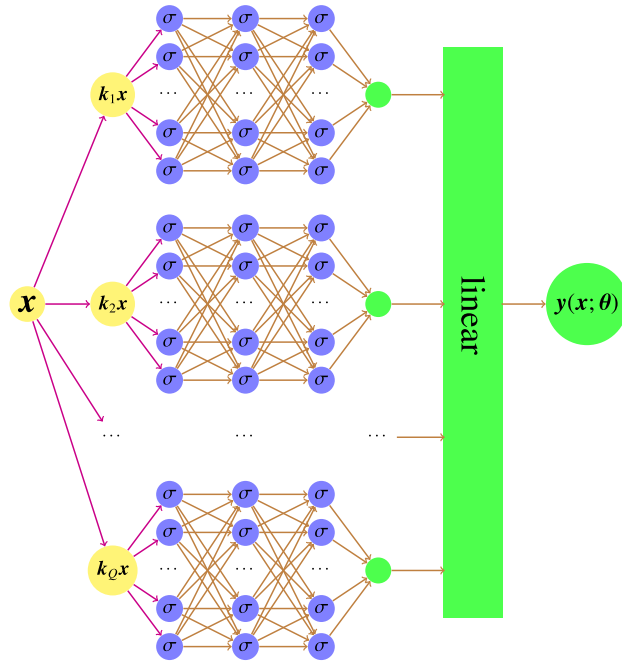


Fig. 1. A schematic diagram of MscaledDNN with Q subnetworks, σ stands for the activation function.

$$\begin{cases} \hat{x} = k_i x \\ F_i(x) = FC\mathcal{N}_i(\hat{x}) \end{cases} \quad i = 1, 2, \dots, Q, \quad (7)$$

where $FC\mathcal{N}_i$ stands for the i_{th} fully connected subnetwork and F_i is its output.

- Obtaining the result of MscaledDNN by aggregating linearly the output of all subnetworks, each scale input goes through a subnetwork. It is

$$NN(x) = W_O \cdot [F_1(x), F_2(x), \dots, F_Q(x)] + b_O, \quad (8)$$

where W_O and b_O stand for the weights and biases of the last linear layer, respectively.

From the perspective of Fourier transformation and decomposition, the first layer of the MscaledDNN model will be treated as a series of basis in Fourier space and its output is the combination of those basis functions [42,44,46].

2.2. Overview of physics-informed neural networks

In the scope of PINN, a type of PDE governed by parameters as the toy to show its implementation, it is

$$\begin{aligned} \mathcal{N}_\lambda[\hat{u}(x)] &= \hat{f}(x), \quad x \in \Omega, \\ \mathcal{B}\hat{u}(x) &= \hat{g}(x), \quad x \in \partial\Omega, \end{aligned} \quad (9)$$

in which $\mathcal{N}_\lambda$ stands for the linear or nonlinear differential operator with parameters λ , $\mathcal{B}$ is the boundary operator, such as Dirichlet, Neumann, periodic boundary conditions, or a mixed form of them. Ω and $\partial\Omega$ respectively illustrate the domain of interest and its boundary. For approximating the solution of the multi-scale PDEs, a multi-scale deep neural network u_{NN} is used. In classical PINN, the ideal parameters of the DNN can be obtained by minimizing the following composite loss function

$$Loss(S_I, S_B; \theta) = Loss_R(S_I; \theta) + \gamma Loss_B(S_B; \theta) \quad (10)$$

with

$$\begin{aligned} Loss_R(S_I; \theta) &= \frac{1}{N_R} \sum_{i=1}^{N_R} \left\| \mathcal{N}_\lambda[u_{NN}(x_i^I, \theta)] - \hat{f}(x_i^I) \right\|^2, \\ Loss_B(S_B; \theta) &= \frac{1}{N_B} \sum_{j=1}^{N_B} \left\| \mathcal{B}u_{NN}(x_j^B, \theta) - \hat{g}(x_j^B) \right\|^2, \end{aligned} \quad (11)$$

where $\gamma > 0$ is used to control the contribution for the corresponding loss term. $Loss_R$ and $Loss_B$ depict the residual of the governing equations and the loss on the boundary condition, respectively. If some additional observed data are available inside the interested domain, then a loss term indicating the mismatch between the predictions produced by DNN and the observations can be taken into account

$$Loss_D = \frac{1}{N_D} \sum_{i=1}^{N_D} \left\| u_{NN}(\mathbf{x}^i, \theta) - u_{Data}^i \right\|^2. \quad (12)$$

Finally, the solution of PDEs is cast into the optimization of DNN. To obtain the ideal θ^* , one can update the weights and biases of DNN through the optimization methods such as gradient descent (GD), stochastic gradient descent (SGD) and the improved versions of SGD including Adam [55], Adagrad [56] and RMSprop [57] during the training process. In this context, the SGD method with a “one-batch” of training data is given by:

$$\theta^{k+1} = \theta^k - \alpha^k \nabla_{\theta^k} Loss(\mathbf{x}; \theta^k) \quad \text{with } \mathbf{x} \in S_I \text{ or } \mathbf{x} \in S_B \quad (13)$$

where the “learning rate” α^k decreases with k increasing.

3. Fourier-based mixed PINN to solve multi-scale problem

In this section, the unified architecture of FMPINN is proposed to overcome the adverse effect of derivative for rough coefficient A^ϵ by embracing a multi-output neural network with an equivalent reduced-order formulation of the multi-scale problem (1).

3.1. Failure of classical PINN

Despite the success of various PINN models in studying ordinary and partial differential equations, it has been observed in Leung et al. [52] that the classical PINN approach fails to provide accurate predictions for multi-scale PDEs (1). Furthermore, we find that a direct application of the PINN with MscaledDNN framework on solving (1) still cannot provide a satisfactory solution, because of the ill-posed NTK matrix caused by rough coefficient A^ϵ . For example, let us consider the following one-dimensional elliptic equation with a homogeneous Dirichlet boundary in $\Omega = [0, 1]$:

$$\begin{cases} -\frac{d}{dx} \left(A^\epsilon(x) \frac{d}{dx} u^\epsilon(x) \right) = 5 \cos(\pi x), \\ u^\epsilon(0) = u^\epsilon(1) = 0, \end{cases}$$

in which $A^\epsilon(x) = \frac{1+x^2}{2+\sin(2\pi x/\epsilon)}$ with $\epsilon > 0$ being a small constant.

We employ the classical PINN method with the MscaledDNN framework (see Fig. 1) to solve (1), called this method as MPINN. The scale factors Λ for MscaledDNN is set as $(1, 2, 3, 4, 5, 10, \dots, 90, 95, 100)$ and the size of each subnetwork is chosen as $(1, 30, 40, 30, 30, 30, 1)$. The activation function of the first hidden layer for all subnetworks is set as Fourier feature mapping (see Section 3.3) and the other activation functions (except for their output layer) are chosen as $\frac{1}{2} \sin(x) + \frac{1}{2} \cos(x)$ [58], their output layers are all linear. For $\epsilon = \frac{1}{32}, \frac{1}{64}$ and $\frac{1}{128}$, we train the aforementioned MPINN model for 50000 epochs and conduct testing every 1000 epochs within the training cycle. The optimizer is set as Adam with an initial learning rate of 0.01 and the learning rate will decay by 2.5% for every 100 epochs. Finally, the results are demonstrated in Fig. 2.

As $\epsilon = 1/32$, the coefficient $A^\epsilon(x)$ possesses a little multi-scale information, and the MPINN performs quite well. However, the permeability $A^\epsilon(x)$ will exhibit various multi-scale properties for $\epsilon = 1/64$, the performance of MPINN deteriorates with a low relative error and the MPINN fails to converge for $\epsilon = 1/128$. In addition, we perform the MPINN with different setups of the hyperparameters such as the learning rate and the γ for $Loss_B$ in (10) as well as the network size, but we still cannot obtain a satisfactory result.

3.2. Unified architecture of FMPINN

Based on the above observation, it is necessary to seek some extra techniques to improve the accuracy of the PINN. Inspired by the mixed finite element method [12,59] and the mixed residual method [27], we can leverage a mixed scheme to solve (1) by replacing the flux term $A^\epsilon \nabla u$ in (1) with an auxiliary variable. This strategy not only can avoid the unfavorable effect of the oscillating coefficient A^ϵ , but also can reduce the computation burden of second-order derivatives in cost function when utilizing a MscaledDNN to approximate the solution of (1). Therefore, we introduce a flux variable $\phi(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_d(\mathbf{x})) = A^\epsilon(\mathbf{x}) \nabla u^\epsilon(\mathbf{x})$ and rewrite the first equation in (1) as the following expressions:

$$\begin{aligned} -\operatorname{div} \phi(\mathbf{x}) &= f(\mathbf{x}), \\ \phi(\mathbf{x}) - A^\epsilon(\mathbf{x}) \nabla u^\epsilon(\mathbf{x}) &= \mathbf{0}. \end{aligned} \quad (14)$$

Then we turn to search a couple of functions (u^ϵ, ϕ) in admissible space with u^ϵ satisfying the prescribed boundary condition, rather than approximating a unique solution of the original problem (1). Here and thereafter, $(u^\epsilon, \phi) \in \mathcal{A} = H^1(\Omega) \times H(\operatorname{div}; \Omega)$ with $H^1(\Omega) = \{v \in L^2(\Omega) : \nabla v \in L^2(\Omega)\}$ and $H(\operatorname{div}; \Omega) = \{\psi \in [L^2(\Omega)]^d : \operatorname{div} \psi \in L^2(\Omega)\}$.

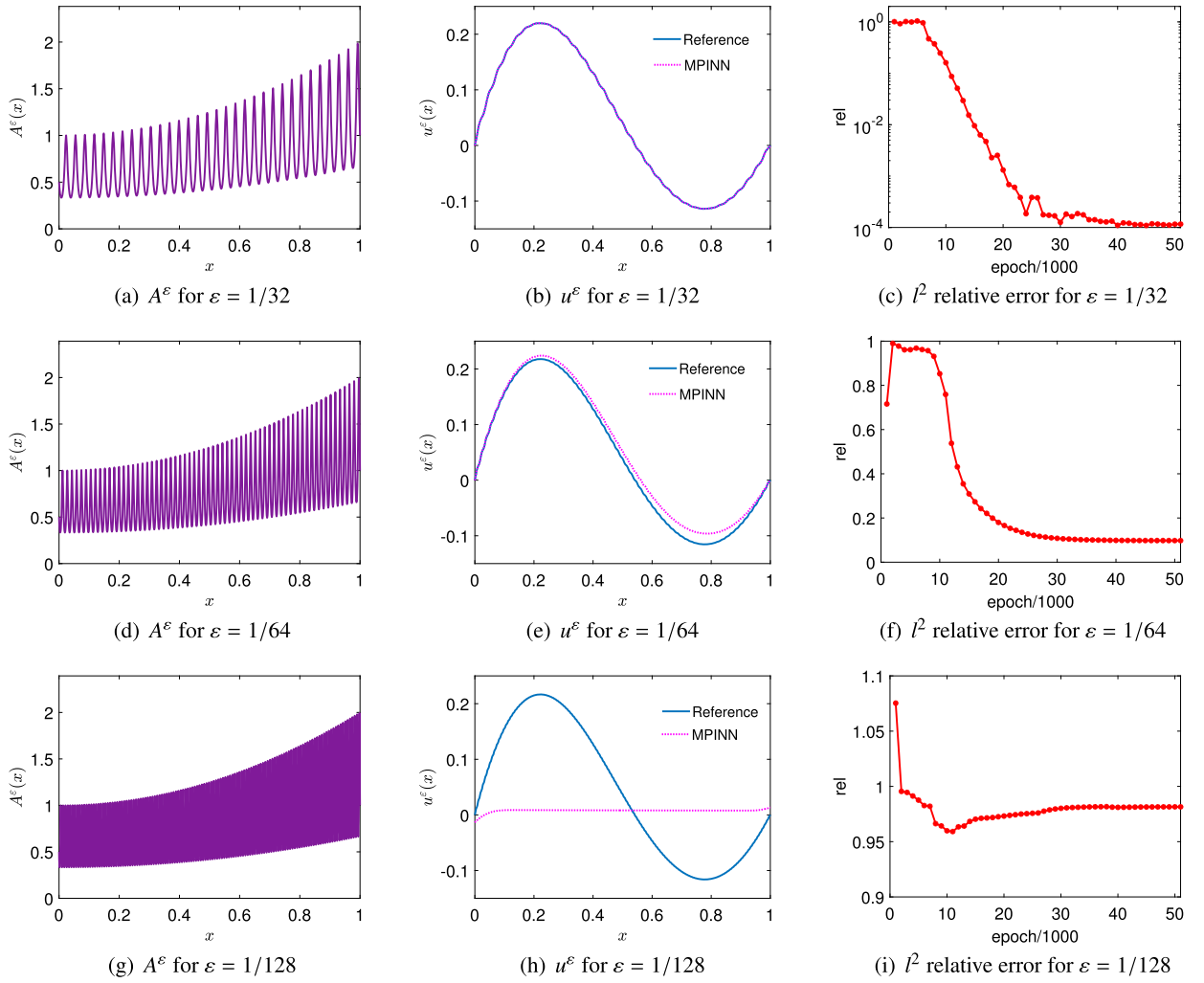


Fig. 2. Left: the rough coefficient A^ϵ . Middle: the MPINN approximated solution vs the reference solution. Right: l^2 relative error varies with the testing epoch. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

When utilizing numerical solvers to address the equation (14), one can obtain the optimum solution by minimizing the following least-squares formula in the domain Ω :

$$u^*, \phi^* = \arg \min_{(u^\epsilon, \phi) \in H^1(\Omega) \times \mathcal{H}(\text{div}; \Omega)} \mathcal{L}(u^\epsilon, \phi) \quad (15)$$

with

$$\mathcal{L}(u^\epsilon, \phi) = \int_{\Omega} \left| -\text{div} \phi(x) - f(x) \right|^2 dx + \beta \int_{\Omega} \left| \phi(x) - A^\epsilon(x) \nabla u^\epsilon(x) \right|^2 dx, \quad (16)$$

where $\beta > 0$ is used to adjust the approximation error of the flux variable and flux term, and u^ϵ is coercive on the boundary of the domain, such as the Dirichlet boundary conditions for the case of a second-order elliptic problem.

Generally, two independent neural networks are necessary to approximate the flux variable ϕ and solution u , but ϕ is unconstrained without any coercive boundary condition. Based on the potentiality of DNN for approximating any linear and non-linear complex functions, we take a DNN with multi outputs to model ansatzes ϕ and u , denoted by ϕ_{NN} and u_{NN} , respectively.

Fig. 3 describes the multi-output neural network for an input $x \in \mathbb{R}^2$.

Once the expressions of auxiliary functions ϕ and solution u^ϵ have been determined, we can discretize (16) by the Monte Carlo method [60], then employ the PINN conception and obtain the following form

$$\mathcal{L}_{in}(S_I; \theta) = \frac{|\Omega|}{N_{in}} \sum_{i=1}^{N_{in}} \left[\left| -\text{div} \phi_{NN}(x_I^i; \theta) - f(x_I^i) \right|^2 + \beta \left| \phi_{NN}(x_I^i; \theta) - A^\epsilon(x_I^i) \nabla u_{NN}(x_I^i; \theta) \right|^2 \right], \quad (17)$$

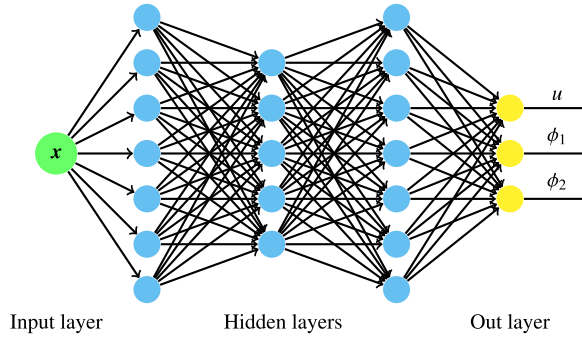


Fig. 3. The multi-output neural network for approximating the state and flux variables.

for $\mathbf{x}_I^j \in S_I$, here and hereinafter S_I stands for the collection sampled from Ω with prescribed probability density.

Same to the traditional numerical methods such as FDM and FEM for addressing PDEs, boundary conditions play a crucial role in DNN representation as well. They serve as important constraints that ensure the uniqueness and accuracy of the solution. Consequently, the output u_{NN} of DNN should also satisfy the boundary conditions of (1), which means

$$\mathcal{L}_{bd}(S_B; \theta) = \frac{1}{N_{bd}} \sum_{j=1}^{N_{bd}} \left[Bu_{NN}(\mathbf{x}_B^j; \theta) - g(\mathbf{x}_B^j) \right]^2 \rightarrow 0 \text{ for } \mathbf{x}_B^j \in S_B, \quad (18)$$

here and hereinafter S_B represents the collection sampled on $\partial\Omega$ with prescribed probability density.

According to the above results, the weights and biases of the DNN model are updated by optimizing gradually the following cost function:

$$\mathcal{L}(S_I, S_B; \theta) = \mathcal{L}_{in}(S_I; \theta) + \gamma \mathcal{L}_{bd}(S_B; \theta), \quad (19)$$

where $S_I = \{\mathbf{x}_I^i\}_{i=1}^{N_{in}}$ and $S_B = \{\mathbf{x}_B^j\}_{j=1}^{N_{bd}}$ stand for the train data of Ω and $\partial\Omega$, respectively. The term of $\mathcal{L}_{in}$ composed of the residual governed by differential equations and the discrepancy concerning flux, minimizes the residual of the PDEs, whereas the term of $\mathcal{L}_{bd}$ pushes the DNN solver to match the given boundary conditions. In addition, a constant parameter $\gamma > 0$ is introduced to force well the $\mathcal{L}_{bd}(S_B; \theta) \rightarrow 0$ in the loss function, it is increasing gradually with training process going on.

Based on the analysis in Bersetche and Borthagaray [39], a nonconstant continuous activation function σ can guarantee the mapping $\theta \mapsto (u_{NN}, \phi_{NN})$ is continuous, then the distance between approximation functions $\mathbf{q}_{NN} = (u_{NN}, \phi_{NN})$ and exact solution $\mathbf{q}^* = (u^*, \phi^*)$ will decrease by adjusting gradually the parameters of DNN, i.e.,

$$d(\mathbf{q}^*, \mathcal{A}_k) = \inf_{\mathbf{q}_{NN} \in \mathcal{A}_k} \|\mathbf{q}^* - \mathbf{q}_{NN}\| \rightarrow 0 \text{ as } k \rightarrow \infty.$$

It means the loss function $\mathcal{L}(S_I, S_B; \theta)$ will attain the corresponding minimum when $d \rightarrow 0$. Hence, Our purpose is to find an optimal set of parameter θ^* such that the approximations u_{NN} and ϕ_{NN} minimize the loss function $\mathcal{L}(S_I, S_B; \theta)$. If the loss function value is small enough, then u_{NN} and ϕ_{NN} will be very close to the solution of (1). Then, optimization methods like SGD, Adam, Adagrad and RMSprop are required to update the parameters of the DNN during the training.

3.3. Option of activation function for FMPINN and its explanation

Choosing a suitable and effective activation function is a critical concern when aiming to enhance the performance of DNN in computer vision, natural language processing, and scientific computation. Generally, an activation function such as rectified linear unit $\text{ReLU}(\mathbf{z})$ and hyperbolic tangent function $\tanh(\mathbf{z})$, can improve the capacity and nonlinearity of neural networks to address various nonlinear problems, such as the solution of various PDEs and classification. Recently, the works [40,41] manifested that the DNN often captures firstly the low-frequency component for target functions, then matches the high-frequency component, they called it the spectral bias or frequency preference of DNN. Under this phenomenon, many researchers attempt to utilize a Fourier feature mapping consisting of sine and cosine as the activation function to improve the capacity of MscaleDNN, it will mitigate the pathology of spectral bias and enable networks to learn high frequencies more effectively [41,46,49,50]. It is expressed as follows:

$$\zeta(\mathbf{x}) = \begin{bmatrix} \cos(\kappa \mathbf{x}) \\ \sin(\kappa \mathbf{x}) \end{bmatrix}, \quad (20)$$

where κ is a user-specified vector or matrix (trainable or untrainable) that is consistent with the number of neural units in the first hidden layer for DNNs. Further, the work [45] designed a soften Fourier mapping by introducing a relaxing parameter $s \in (0, 1]$ in $\zeta(\mathbf{x})$, numerical results show that this modification will improve the capacity of $\zeta(\mathbf{x})$. This activation function is used in the first hidden layer of DNN and maps the input data in Ω into a range of $[-1, 1]$, then enhances the ability of DNN and expedites its convergence.

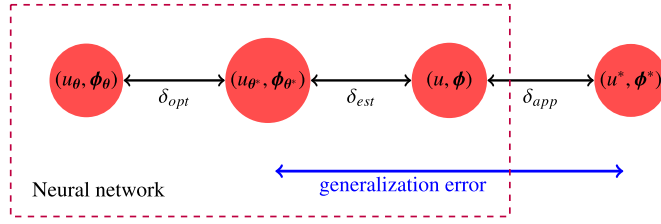


Fig. 4. Illustration of the total error for FMPINN.

Therefore, a real function $\mathcal{P}(x)$ represented by DNN can be expressed as follows

$$\mathcal{P}(x) = \sum_{n=0}^{\tilde{N}} \left(S(\cos(k_n x); \bar{\theta}_n) + T(\sin(k_n x); \bar{\theta}_n) \right),$$

where $S(\cdot, \bar{\theta}), T(\cdot, \bar{\theta})$ are the DNNs or the sub-modules of DNNs, respectively, $\{k_0, k_1, k_2, \dots\}$ are the frequencies of interest for the objective function. The first hidden layer performed by Fourier feature mapping mimics the Fourier basis function, and the remaining blocks with different activation functions are used to learn the coefficients of these functions. After performing the Fourier mapping for input points with a given scale factor, the neural network can well capture the fine varying information for multi-scale problems.

Remark 1. (Lipschitz continuity) If an activation function σ is continuous (i.e., $\sigma \in C^1$) and satisfies the following boundedness condition:

$$|\sigma(x)| < 1 \quad \text{and} \quad |\sigma'(x)| < 1,$$

for any $x \in \mathbb{R}$. Then, we have

$$|\sigma(x) - \sigma(y)| \leq |x - y| \quad \text{and} \quad |\sigma'(x) - \sigma'(y)| < |x - y|,$$

for any $x, y \in \mathbb{R}$. The activation functions $\tanh(x)$, $\text{sigmoid}(x)$, Fourier feature mapping $\zeta(x)$ and $\frac{1}{2}\sin(x) + \frac{1}{2}\cos(x)$ are all satisfy the above condition and have a good regularity, they will overcome the gradient explosion of parameter in the backpropagation for DNN and improve the capacity of DNN.

3.4. Simple error analysis for FMPINN

In recent times, there have been endeavors to rigorously analyze the convergence rate of the deep mixed residual method and compare it with the deep Galerkin method (DGM) and deep Ritz method (DRM) across different scenarios [39,61,62]. In this study, we investigate those results of convergence again, then provide the expression of generalization error for FMPINN and some remarks of errors.

For convenience, let $q^* = (u^*, \phi^*)$ be the exact solution of equation (14) or the minimum of cost function (15) with (16) for coercive boundary constraints. Meantime, the $q_{\theta^*} = (u_{\theta^*}, \phi_{\theta^*})$ stands for the final output of DNN optimized by stochastic optimizer (such as Adam or SGD) that attains the local minimum of (19). Further, we let $\widetilde{\mathcal{L}}(u, \phi)$ be the cost function evaluated on N points sampled from Ω and denote the output of DNN as $q_{\theta} = (u_{\theta}, \phi_{\theta})$. Finally, S_{NN} represents the function space spanned by the output of DNN. Then, the total error (or generalization error) between the exact solution q^* and the output of DNN q_{θ} can be expressed as

$$\|u_{\theta} - u^*\|_{H^1(\Omega)} + \|\phi_{\theta} - \phi^*\|_{H^1(\text{div}, \Omega)} \leq C(\text{coe}) \sqrt{\delta_{app} + \delta_{est} + \delta_{opt}} \quad (21)$$

with

$$\begin{cases} \delta_{app} = \inf_{(u, \phi) \in S_{NN}} \|u - u^*\|_{H^1(\Omega)}^2 + \|\phi - \phi^*\|_{H^1(\text{div}, \Omega)}^2, \\ \delta_{est} = \sup_{(u, \phi) \in S_{NN}} [\mathcal{L}(u, \phi) - \widetilde{\mathcal{L}}(u, \phi)] + \sup_{(u, \phi) \in S_{NN}} [\widetilde{\mathcal{L}}(u, \phi) - \mathcal{L}(u, \phi)], \\ \delta_{opt} = \widetilde{\mathcal{L}}(u_{\theta^*}, \phi_{\theta^*}) - \widetilde{\mathcal{L}}(u_{\theta}, \phi_{\theta}). \end{cases}$$

In which, the approximated error δ_{app} indicates the difference between (u^*, ϕ^*) and its projection onto S_{NN} , the estimation error δ_{est} measures the difference between the continuous cost function $\mathcal{L}$ and discrete cost function $\widetilde{\mathcal{L}}$, the optimization error δ_{opt} stands for the discrepancy between the output of DNN with optimizing and the output of DNN without optimizing. In Fig. 4, we depict the diagram of error for FMPINN. The convergence and error analysis of MIM in works [61,62] can be used to support our results in this section by appropriately adjusting their corresponding conclusions.

Remark 2. The approximating error is generally dependent on the architectural design of the neural network and the choice of the activation function. Classical radial basis network [63], the vanilla DNN and Random feature network [64] as well as its special

case Extreme Learning Machine (ELM) [65,66] are the common meshless method for approximating the solution of PDEs. To address the spatio-temporal problems, some hybrid network frameworks have been designed by combining PINN with traditional numerical methods to solve PDEs, such as FDM-PINN and Runge-Kutta PINN [24,67]. Moreover, instead of soft constraints in a hard manner for the boundary or initial conditions in those methods, the approximation will automatically meet the boundary and initial conditions of PDEs, then reduce the complexity and improve the precision of NN [35]. On the other hand, a powerful activation function, such as the hyperbolic tangent activation function and Fourier feature mapping, not only enhances the nonlinearity of DNN but also improves its approximating capacity and accuracy. In addition, some available data are generally considered as a loss term to reduce the approximating error.

Remark 3. Generally, the proposed FMPINN surrogate can provide more accurate approximations as the number of random collocation points increases. However, it will lead to heavy computational costs for lots of samplings. Then, it is worthwhile to take into account the trade-off between accuracy and computational cost when designing a DNN surrogate and determining its training mode. Alternatively, one can employ some effective low-discrepancy sampling approaches to decrease the statistical error, such as the Latin hypercube sampling method [68], quasi-random sampling [69] and multilevel Monte Carlo method [70].

Remark 4. Since the cost function generally is non-convex and has several local minima, then the gradient-based optimizer will almost certainly become caught in one of them. Therefore, choosing a good optimizer is important to reduce the optimization error and get a better minimum. In many scenarios of optimizing DNN, the Adam optimization method has shown its good performance including efficiency and accuracy, it can dynamically adjust the learning rates of each parameter by using the first and second moments estimation of the gradients [55]. BFGS is a quasi-Newton method and numerically stable, it may provide a higher-precision approximated solution [71]. In an implementation, the limited memory version of BFGS (L-BFGS) is the common choice to decrease the optimization error and accelerate convergence for cases with a small amount of training data and/or residual points. Further, by combining the merits of the above two approaches, one can optimize the cost function firstly by the Adam algorithm with a predefined stop criterion, then obtain a better result by the L-BFGS optimizer.

4. FMPINN algorithm

For the FMPINN method with the MscaledDNN model composed of Q subnetworks as in Fig. 1 being its solver, the input data for each subnetwork will be transformed by the following operation

$$\hat{\mathbf{x}} = a_i * \mathbf{x}, \quad i = 1, 2, \dots, Q,$$

with $a_i \geq 1$ being a positive scalar factor, it means the scale vector $\Lambda = (a_1, a_2, \dots, a_Q)$ as in (6). Denoting the output of each subnetwork as $F_i (i = 1, 2, \dots, Q)$, then the overall output of the MscaledDNN model is obtained by

$$y(\mathbf{x}; \theta) = \frac{1}{Q} \sum_{i=1}^Q \frac{F_i}{a_i}.$$

According to the above discussions, the procedure of the FMPINN algorithm for addressing the multi-scale problem (1) in finite-dimensional spaces is described in the following.

Algorithm 1 FMPINN algorithm for solving multi-scale PDEs (1)

1. Generating the k_{th} training set S^k includes interior points $S_I^k = \{\mathbf{x}_I^i\}_{i=1}^{N_{in}}$ with $\mathbf{x}_I^i \in \mathbb{R}^d$ and boundary points $S_B^k = \{\mathbf{x}_B^j\}_{j=1}^{N_{bd}}$ with $\mathbf{x}_B^j \in \mathbb{R}^d$. Here, we draw the random points $\mathbf{x}_I^i$ and $\mathbf{x}_B^j$ from $\mathbb{R}^d$ with positive probability density v , such as uniform distribution.
2. Calculating the objective function $\mathcal{L}(S^k; \theta^k)$ for training set S^k :

$$\mathcal{L}(S^k; \theta^k) = \mathcal{L}_{in}(S_I^k; \theta^k) + \gamma \mathcal{L}_{bd}(S_B^k; \theta^k)$$

with $\mathcal{L}_{in}(\cdot; \theta^k)$ being defined in (17) and $\mathcal{L}_{bd}(\cdot; \theta^k)$ being defined in (18).

3. Taking a suitable optimization method to update the internal parameters of DNN at the random point of $\bar{\mathbf{x}}^k$, such as SGD method expressed as follows:

$$\theta^{k+1} = \theta^k - \alpha^k \nabla_{\theta^k} \mathcal{L}(\bar{\mathbf{x}}^k; \theta^k) \quad \text{with} \quad \bar{\mathbf{x}}^k \in S^k,$$

where the “learning rate” α^k decreases with k increasing.

4. Repeating steps 1-3 until the convergence criterion is satisfied or the objective function tends to be stable.
-

5. Numerical experiments

The goal of our experiments is to show that our FMPINN is indeed capable of approximating the analytical solution given in (1). For comparison purposes, the PINN method with MscaledDNN being its solver and the local deep learning method (LDLM) with normal DNN being its solver is as the baseline to solve (1) in varying-dimensional spaces.

5.1. Model and training setup

In the aforementioned FMPINN and MPINN models, a standard MscaleDNN with multi subnetworks that stretch the input data via various scale factors is configured as their solver. The MscaleDNN consists of 24 subnetworks according to the manually defined frequencies vector $\Lambda = (1, 2, 3, 4, 5, 10, \dots, 90, 95, 100)$. Each subnetwork contains 5 hidden layers with proper size and the activation function of the first hidden layer for each subnetwork is set as Fourier feature mapping and the other activation functions (except for the output layer) are set as $\frac{1}{2} \sin(x) + \frac{1}{2} \cos(x)$, its output layer is linear. The overall output is a weighted sum of the outputs of all subnetworks through the relevant scale factors. In terms of the LDLM [37], two activation functions are considered for this model: LDLM1 with $ReLU = \max\{0, x\}^2$ being its activation for hidden layers and LDLM2 with $\frac{1}{2} \sin(x) + \frac{1}{2} \cos(x)$ being its activation function for hidden layers, their output is all linear.

In our numerical experiments, all training data are sampled from the domain (including its boundaries) of interest in Euclidean space $\mathbb{R}^d$, the sampling probability densities are assigned as the uniform distribution. We train all neural networks by an Adam optimizer with an initial learning rate of 0.01, and the learning rate will be decayed by 2.5% for every 100 training epochs [55]. Here, the following L^2 relative error is used to evaluate our models:

$$REL = \sqrt{\frac{\sum_{i=1}^{N'} |\tilde{u}(\mathbf{x}^i) - u^*(\mathbf{x}^i)|^2}{\sum_{i=1}^{N'} |u^*(\mathbf{x}^i)|^2}},$$

where $\tilde{u}(\mathbf{x}^i)$ and $u^*(\mathbf{x}^i)$ are the approximate solution of deep neural network and exact solution for testing points $\{\mathbf{x}^i\} (i = 1, 2, \dots, N')$, respectively, and N' represents the number of sample points for testing. In addition, we also introduce the following relative error within the H^1 norm to measure the approximation accuracy for our proposed FMPINN.

$$H^1 ERR = \sqrt{\frac{\sum_{i=1}^{N'} |\tilde{u}(\mathbf{x}^i) - u^*(\mathbf{x}^i)|^2 + \sum_{i=1}^{N'} |\nabla \tilde{u}(\mathbf{x}^i) - \nabla u^*(\mathbf{x}^i)|^2}{\sum_{i=1}^{N'} |u^*(\mathbf{x}^i)|^2 + \sum_{i=1}^{N'} |\nabla u^*(\mathbf{x}^i)|^2}}$$

To visualize the training process, our model will be evaluated once for every 1000 iterations in the whole training cycle and recorded the result at the end. In our codes, the penalty parameter γ is set as

$$\gamma = \begin{cases} \gamma_0, & \text{if } i_{\text{epoch}} < M_{\max} * 0.1 \\ 10\gamma_0, & \text{if } M_{\max} * 0.1 \leq i_{\text{epoch}} < M_{\max} * 0.2 \\ 50\gamma_0, & \text{if } M_{\max} * 0.2 \leq i_{\text{epoch}} < M_{\max} * 0.25 \\ 100\gamma_0, & \text{if } M_{\max} * 0.25 \leq i_{\text{epoch}} < M_{\max} * 0.5 \\ 200\gamma_0, & \text{if } M_{\max} * 0.5 \leq i_{\text{epoch}} < M_{\max} * 0.75 \\ 500\gamma_0, & \text{otherwise,} \end{cases} \quad (22)$$

where the $\gamma_0 = 10$ in all our tests and $M_{\max}$ represents the total number of epochs. We implement and perform all neural network models using the package of Pytorch (version 1.14.0) on a workstation (64 GB RAM, single NVIDIA GeForce RTX 4090 24-GB).

5.2. Performance of FMPINN for solving multi-scale elliptic PDEs

Example 1. Firstly, we consider the one-dimensional case for (1) with Dirichlet boundary in interval $[0, 1]$, in which $A^\varepsilon(x)$ is given by

$$A^\varepsilon(x) = \left(2 + \cos\left(2\pi \frac{x}{\varepsilon}\right)\right)^{-1} \quad (23)$$

with a small parameter $\varepsilon > 0$ such that $\varepsilon^{-1} \in \mathbb{N}^+$ and the force term $f(x) = 1$. Under these conditions, a unique solution is given by

$$u^\varepsilon(x) = x - x^2 + \varepsilon \left(\frac{1}{4\pi} \sin\left(2\pi \frac{x}{\varepsilon}\right) - \frac{1}{2\pi} x \sin\left(2\pi \frac{x}{\varepsilon}\right) - \frac{\varepsilon}{4\pi^2} \cos\left(2\pi \frac{x}{\varepsilon}\right) + \frac{\varepsilon}{4\pi^2} \right). \quad (24)$$

The analytical solution induces its boundary condition $u^\varepsilon(0) = u^\varepsilon(1) = 0$.

In this example, we use the FMPINN, MPINN, LDLM1, and LDLM2 models to solve (1) when $\varepsilon = 0.1, 0.01$ and 0.001 , respectively. The size of the hidden layer for each subnetwork of FMPINN and MPINN is set as (30, 40, 30, 30, 30) and the balance parameter β in (17) is set as 10. The hidden layer's size for LDLM is set as (300, 400, 300, 300, 300). Their parameters' numbers are comparable. At each training step, we randomly sample 3000 points inside the $[0, 1]$ and 500 boundary points as a training dataset. In addition, the testing dataset includes 1000 equidistant samples from $[0, 1]$. All models are trained for 50000 epochs. We depict the related experiment results in Figs. 5, 6 and 7, respectively. Meantime, the final relative L^2 and H^1 error and total running time are listed in Tables 1 and 2.

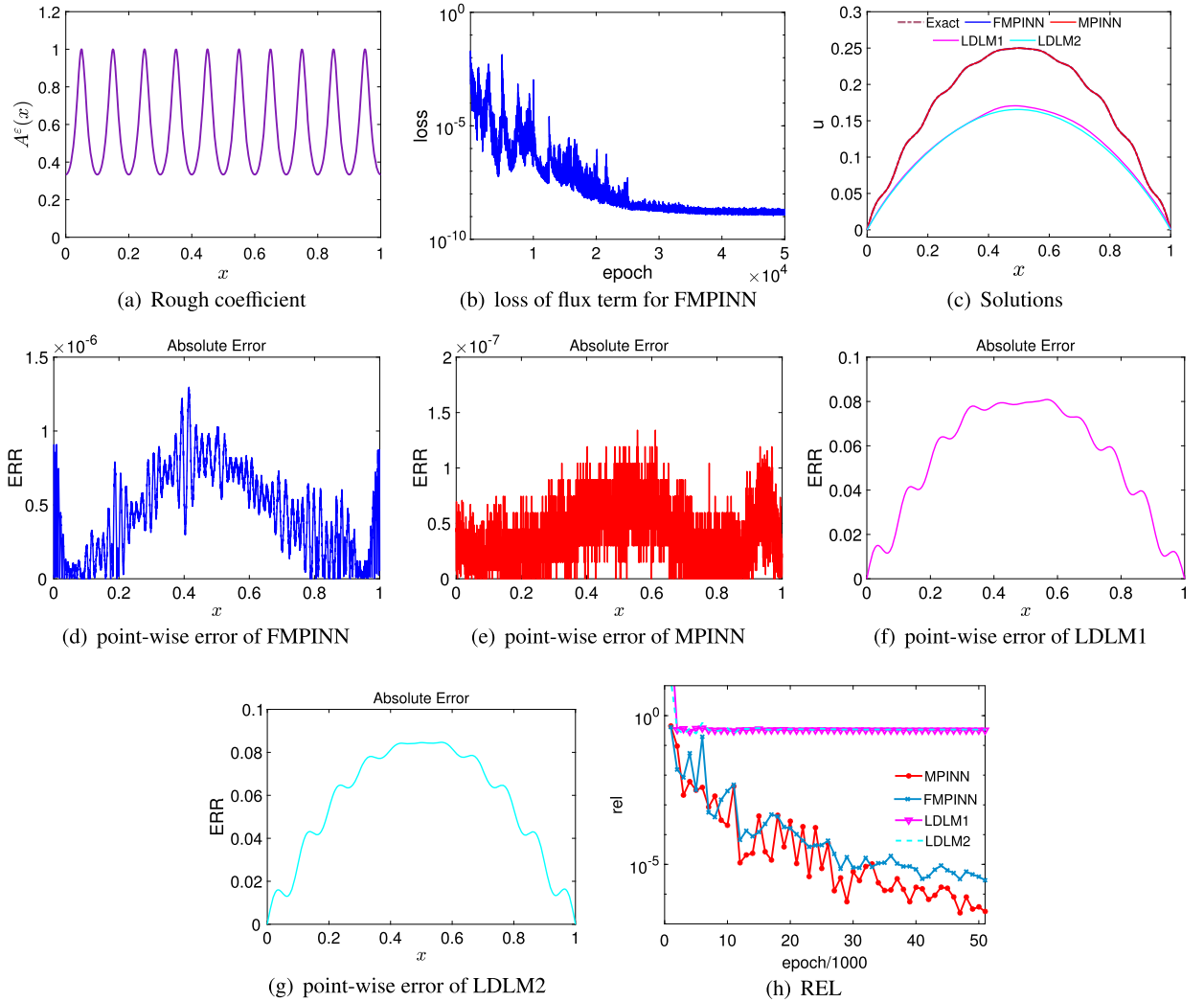


Fig. 5. Rough coefficient, loss of flux term and testing results for Example 1 when $\varepsilon = 0.1$.

Table 1

The relative error and running time of FMPINN, MPINN, LDLM1, and LDLM2 for Example 1.

ε	REL				Total time(s)			
	FMPINN	MPINN	LDLM1	LDLM2	FMPINN	MPINN	LDLM1	LDLM2
0.1	2.92e-6	2.60e-7	0.3227	0.3389	680.734	865.849	345.791	373.537
0.01	1.21e-6	0.874	0.3397	0.3406	689.729	868.199	351.451	377.089
0.001	9.28e-5	0.99	0.3389	0.3398	691.458	875.297	358.435	388.273

Based on these figures, the FMPINN model can perfectly capture the oscillation of the exact solution for $\varepsilon = 0.1, 0.01$ and 0.001 , but LDLM models are not convergent for these cases. At the same time, the performance of MPINN competes with that of FMPINN when $\varepsilon = 0.1$. However, the MPINN model fails to solve the multi-scale problem for $\varepsilon = 0.1$ and 0.01 . Compared to $\varepsilon = 0.01$, the rough coefficient A^ε with $\varepsilon = 0.001$ have more oscillation in the interval $[0, 1]$, but the FMPINN still can keep its remarkable performance. According to the point-wise errors in Figs. 5(d), 6(d) and 7(d) and the relative error in Figs. 5(h), 6(h) and 7(h), we can conclude that the FMPINN can approximate high-precisely the exact solution of (1) in one-dimensional space. In addition, the total time in Table 1 shows the running time of FMPINN is less than that of MPINN for 50000 training epochs.

In addition, the finite element method (FEM) is introduced for comparison in solving the multiscale problem (1) in one-dimensional space with varying numbers of equidistant points. The results of both FMPINN and FEM methods are presented in Table 3, which demonstrate that our proposed method surpasses the FEM in terms of accuracy for 1000 equidistant points within the interval $[0, 1]$. With the number of points increasing, the performance of FEM will become better and better for $\varepsilon = 0.1, 0.01$, and 0.001 , and it will surpass FMPINN in terms of accuracy and running time for 100000 and 500000 equidistant points.

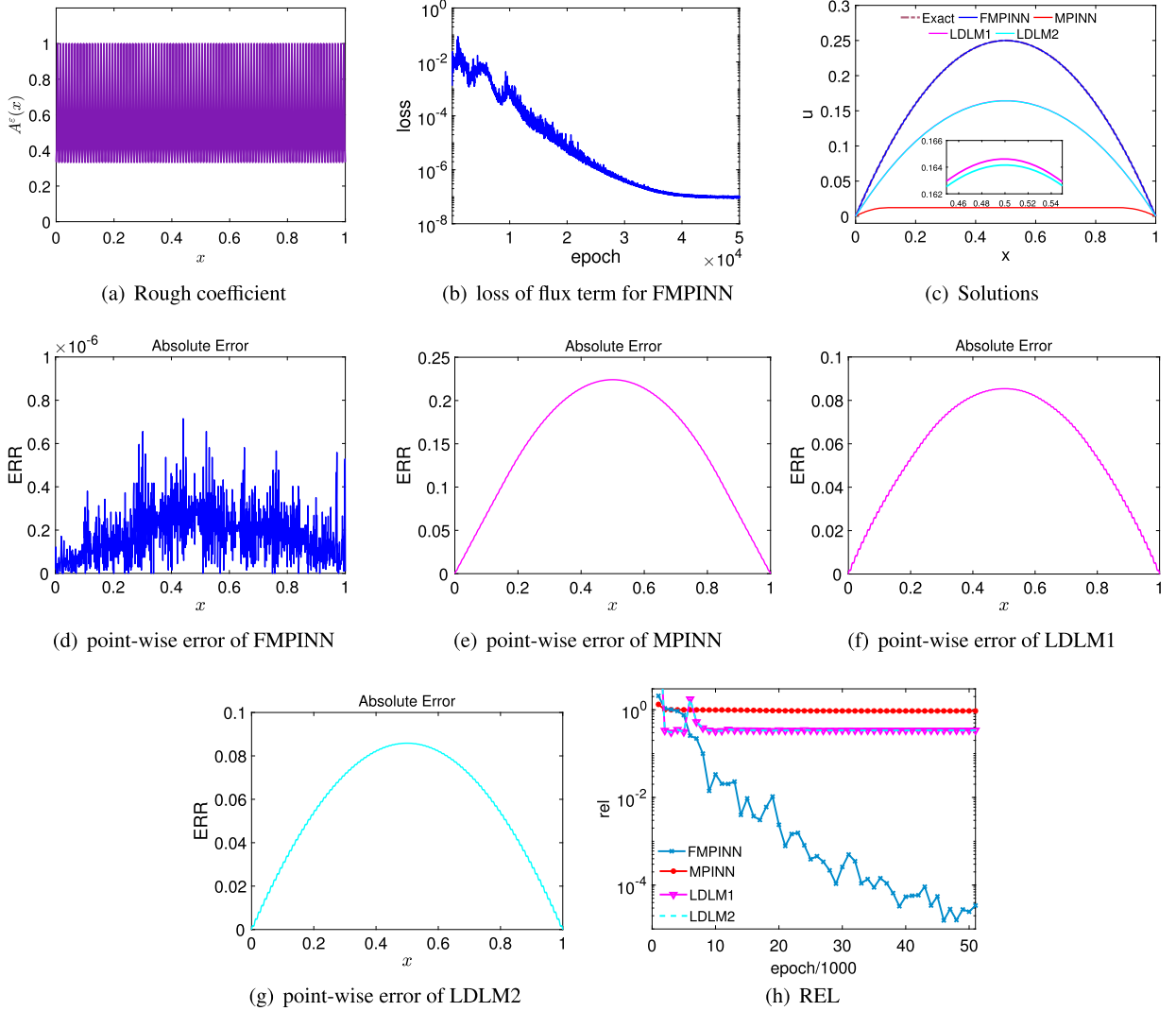


Fig. 6. Rough coefficient, loss of flux term and testing results for Example 1 when $\varepsilon = 0.01$.

Table 2

The relative error in H^1 norm for FMPINN model VS various ε for Example 1.

ε	0.1	0.01	0.001
H^1 ERR	0.01753	0.001823	0.002995

Table 3

The REL and running time of FMPINN and Finite Element Method for Example 1.

Method	Number of points	REL			Time(s)		
		$\varepsilon = 0.1$	$\varepsilon = 0.01$	$\varepsilon = 0.001$	$\varepsilon = 0.1$	$\varepsilon = 0.01$	$\varepsilon = 0.001$
FMPINN	1000	2.922e-6	1.215e-6	9.283e-5	680.734	689.729	691.458
FEM	1000	4.447e-5	0.0044	0.1338	0.3365	0.3057	0.3168
	5000	1.776e-6	1.762e-4	0.01666	0.7994	0.7834	0.7162
	10000	4.439e-7	4.408e-5	0.0043	1.3435	1.2936	1.3011
	50000	1.751e-8	1.763e-6	1.762e-4	5.6383	5.8812	5.7875
	100000	5.831e-9	4.416e-7	4.407e-5	11.1615	11.1076	11.8971
	500000	1.175e-8	3.162e-8	1.757e-6	53.2835	53.1815	53.4899

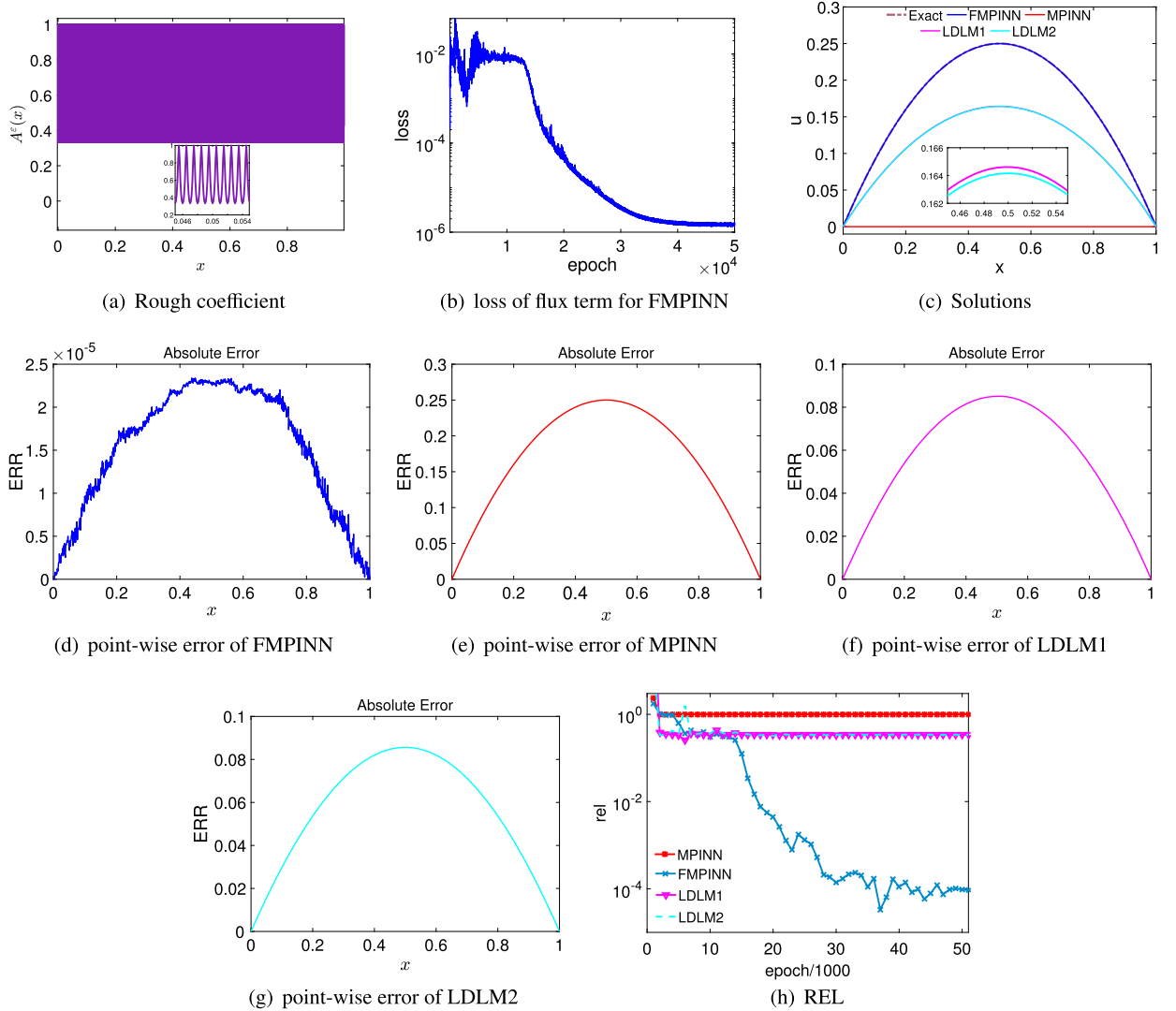


Fig. 7. Rough coefficient, loss of flux term and testing results for Example 1 when $\varepsilon = 0.001$.

Table 4

The relative error of FMPINN model VS various β for Example 1.

β	1	5	10	15	20	25
REL	3.31e-4	1.37e-4	9.27e-5	7.75e-5	6.40e-5	8.60e-5

Influence of hyper-parameter β : In the previous tests, the parameter β was initially set to 10. Now, we study the influence of β for our FMPINN model. In these tests, we take $\varepsilon = 0.001$ in (23) and consider values of β equal to 1, 5, 10, 15, 20 and 25, while keeping all other parameters fixed. All models with different β values are trained for 50000 epochs. Fig. 8 plots the results of flux loss for the training process as well as the relative error for testing. Additionally, the final relative errors obtained from the tests are listed in Table 4.

According to the above results in Fig. 8 and Table 4, it can be observed that the FMPINN model exhibits remarkable and stable performance across different values of β . The performances of the FMPINN model for $\beta = 1$ and $\beta = 5$ are slightly weaker than that of other cases. The loss of flux term is also stable and consistent with the trendlines of REL. Therefore, for the subsequent tests, we will continue to set $\beta = 10$.

Influence of the activation function for FMPINN: Now, we study the influence of the activation function for our proposed FMPINN model to solve (1). In the test, the activation functions of the hidden layers (except the first layer and the last layer) for each subnetwork in the FMPINN model are set as sin, tanh, requ, elu, gelu, relu, and leaky_relu, respectively, and the other setups are identical to the above experiments. Based on the results in Table 5, we observe that the performance of $\frac{1}{2} \sin(x) + \frac{1}{2} \cos(x)$ and gelu

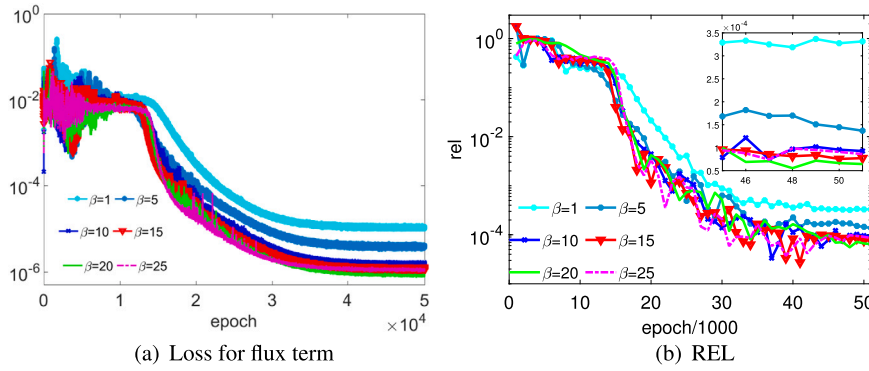


Fig. 8. The loss of flux term VS training epoch and the relative error VS testing epoch for Example 1 when $\varepsilon = 0.001$.

Table 5

REL of FMPINN with different activation functions for solving Example 1 with $\varepsilon = 0.01$ and $\varepsilon = 0.001$.

ε	sin	tanh	$\frac{1}{2}\sin(x) + \frac{1}{2}\cos(x)$	requ	elu	gelu	relu	leaky relu
0.1	0.0014	5.83e-6	2.92e-6	9.66e-4	2.33e-6	8.22e-6	5.52e-4	0.0019
0.01	0.99	1.52e-4	1.21e-6	0.0783	4.28e-4	4.11e-6	0.79	0.27
0.001	0.99	0.31	9.28e-5	0.337	0.339	0.338	0.94	0.34

Table 6

The REL and running time of FMPINN with different optimizers for Example 1 when $\varepsilon = 0.01$.

Optimizer	SGD	Moment	Adam	LBFGS	Adam+LBFGS	LBFGS+Adam
REL	0.3449	0.3421	1.21e-6	0.0014	1.18e-6	8.96e-5
Time(s)	635.99	640.86	656.49	4624.38	1040.22	2705.12

are superior to that of others for our FMPINN model when $\varepsilon = 0.1$ and $\varepsilon = 0.01$, and their performance compete. However, the FMPINN models with the above activation functions all failed in approximating the solution of multi-scale problem (1). Then, the activation function $\frac{1}{2}\sin(x) + \frac{1}{2}\cos(x)$ is an ideal candidate for our FMPINN model.

Influence of the optimizer for FMPINN: In the test, all models with different optimizers are trained for 50000 epochs. Except for the original Adam optimizer for our FMPINN model, we further set the optimizer as SGD, Momentum, Adam, LBFGS, Adam+LBFGS, and LBFGS+Adam, respectively, their setups are as follows:

- **SGD:** the corresponding setups are same as the Adam optimizer;
- **Momentum:** the initial learning rate and the decay schedule are the same as the Adam optimizer, its momentum is set as 0.5;
- **LBFGS:** the initial learning rate and the decay schedule are the same as the Adam optimizer, its max iteration, history size of the change, and line search function are set as 500, 100, and strong Wolfe, respectively;
- **Adam+LBFGS:** the FMPINN model is trained using Adam with an initial learning rate of 0.01 for 40000 epochs, then trained using LBFGS with an initial learning rate of 0.0001 for 10000 epochs. Their other setups are the same as the Adam and LBFGS in this test;
- **LBFGS+Adam:** the FMPINN model is trained using LBFGS with an initial learning rate of 0.01 for 10000 epochs, then trained using Adam with an initial learning rate of 0.001 for 40000 epochs. Their other setups are the same as the Adam and LBFGS in this test.

The other setups are identical to the above experiments. Based on the results in Table 6 and Fig. 9, we find that FMPINN models utilizing Adam and Adam+LBFGS optimizers outperform those employing other optimizers. Furthermore, the running time for FMPINN models with Adam is less than that with Adam+LBFGS. Therefore, Adam appears to be an ideal candidate optimizer for our FMPINN model.

Example 2. Let us attempt to solve the following three-scale problem with the Dirichlet boundary in $\Omega = [0, 1]$. In which,

$$A^\varepsilon(x) = \left(2 + \cos\left(2\pi \frac{x}{\varepsilon_1}\right)\right) \left(2 + \cos\left(2\pi \frac{x}{\varepsilon_2}\right)\right)$$

with two small parameters $\varepsilon_1, \varepsilon_2 > 0$ such that $\varepsilon_1^{-1}, \varepsilon_2^{-1} \in \mathbb{N}^+$ and an exact solution is given by

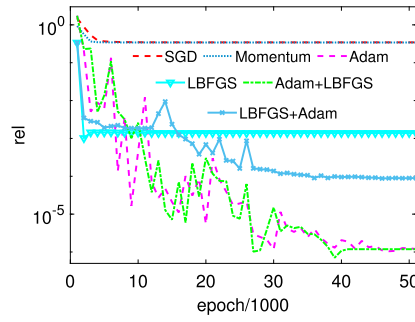


Fig. 9. The REL curves for FMPINN model with different optimizers.

Table 7

The relative error and consumed time of FMPINN, MPINN, LDLM1, and LDLM2 for Example 2.

Method	FMPINN	MPINN	LDLM1	LDLM2
REL	3.36e-4	5.02e-2	0.5341	0.8372
H ¹ ERR	0.0121	0.0423	0.965	0.989
Total time(s)	696.537	965.076	377.18	399.575

$$u^\varepsilon(x) = x - x^2 + \frac{\varepsilon_1}{4\pi} \sin\left(2\pi \frac{x}{\varepsilon_1}\right) + \frac{\varepsilon_2}{4\pi} \sin\left(2\pi \frac{x}{\varepsilon_2}\right). \quad (25)$$

Clearly, $u^\varepsilon(0) = u^\varepsilon(1) = 0$. One can obtain the force side after careful computation, we omit it here.

We solve the above three scale problems when $\varepsilon_1 = 0.1$ and $\varepsilon_2 = 0.01$ by employing the aforementioned FMPINN, MPINN, LDLM1, and LDLM2 models, respectively. All settings are the same as the Example 1. The training dataset includes 3000 interior random points and 500 boundary random points, and the testing dataset includes 1000 equidistant samples. The related experiment results are listed in Table 7 and plotted in Fig. 10, respectively.

Fig. 10 shows that the FMPINN model still is well able to capture all oscillations of the exact solution for the three-scale problem, the MPINN model also captures the profile of the solution of (1) with $\varepsilon_1 = 0.1$ and $\varepsilon_2 = 0.01$. However, the LDLM1 and LDLM2 all fail to fit the solution. Figs. 10(d) – 10(g) not only show the point-wise errors of FMPINN for major points that are close to zero but also reveal the point-wise error of FMPINN is very smaller than that of the MPINN and the LDLM models are all bad. Additionally, Fig. 10(h) and Table 7 illustrate that the REL of FMPINN is superior to that of MPINN by more than two orders of magnitude, and the former H¹ERR is approximately one-quarter of the latter. In addition, its running time is 696.537 seconds and less than that of MPINN.

From the above results, we conclude that the FMPINN model is remarkable to address the (1) with rough coefficient in one-dimensional space, it generally outperforms the MPINN and LDLM models.

Example 3. We consider the following two-dimensional problem for (1) with Dirichlet boundary in regular domains $\Omega = [-1, 1] \times [-1, 1]$. In this example, we choose the $f(x_1, x_2) = 5$ and provide the following two-scales coefficient with scale separation

$$A^\varepsilon(x_1, x_2) = \frac{1.5 + \sin(2\pi x_1/\varepsilon)}{1.5 + \sin(2\pi x_2/\varepsilon)} + \frac{1.5 + \sin(2\pi x_2/\varepsilon)}{1.5 + \cos(2\pi x_1/\varepsilon)} + \sin(4x_1^2 x_2^2) + 1, \quad (26)$$

where $\varepsilon > 0$ is a small parameter such that $\varepsilon^{-1} \in \mathbb{N}^+$. Since the corresponding exact solution can not be expressed explicitly in this example, then a reference solution $u^\varepsilon(x_1, x_2)$ is set as the finite element solution computed by numerical homogenization method [15] on a square grid $[-1, 1] \times [-1, 1]$ with mesh-size $h = 1/128$.

We solve the above two scale problems when $\varepsilon = 0.05$ by employing the aforementioned FMPINN, MPINN, LDLM1, and LDLM2 models, respectively. The size of the hidden layer for each subnetwork of FMPINN and MPINN is set as (40, 60, 40, 40, 40) and the hidden layers' size for LDLMs is set as (400, 250, 250, 200, 200). At each training step, the training dataset includes 5000 points sampled inside the Ω and 2000 boundary points sampled from the $\partial\Omega$, respectively. To test our models, the testing dataset is the collection of all grid points in domain $[-1, 1] \times [-1, 1]$ with mesh-size $h = 1/128$. The related experiment results are listed in Table 8 and plotted in Fig. 11, respectively.

In this example, the $A^\varepsilon(x_1, x_2)$ have two different frequency components and is quite oscillating (see Fig. 11(a)), then DNN will encounter some troubles to address multi-scale PDEs (1). According to the results of point-wise error (Figs. 11(d) - 11(g)) and relative errors (Fig. 11(h)), the performance of our FMPINN model is still superior to that of the MPINN, LDLM1 and LDLM2 models, and can obtain a favorable approximation to multi-scale problems (1). In addition, the test REL curve in Fig. 11(h) indicates the FMPINN model is stable in the whole training cycle and its tendency is consistent with the curve of loss for flux term in Fig. 11(c). The

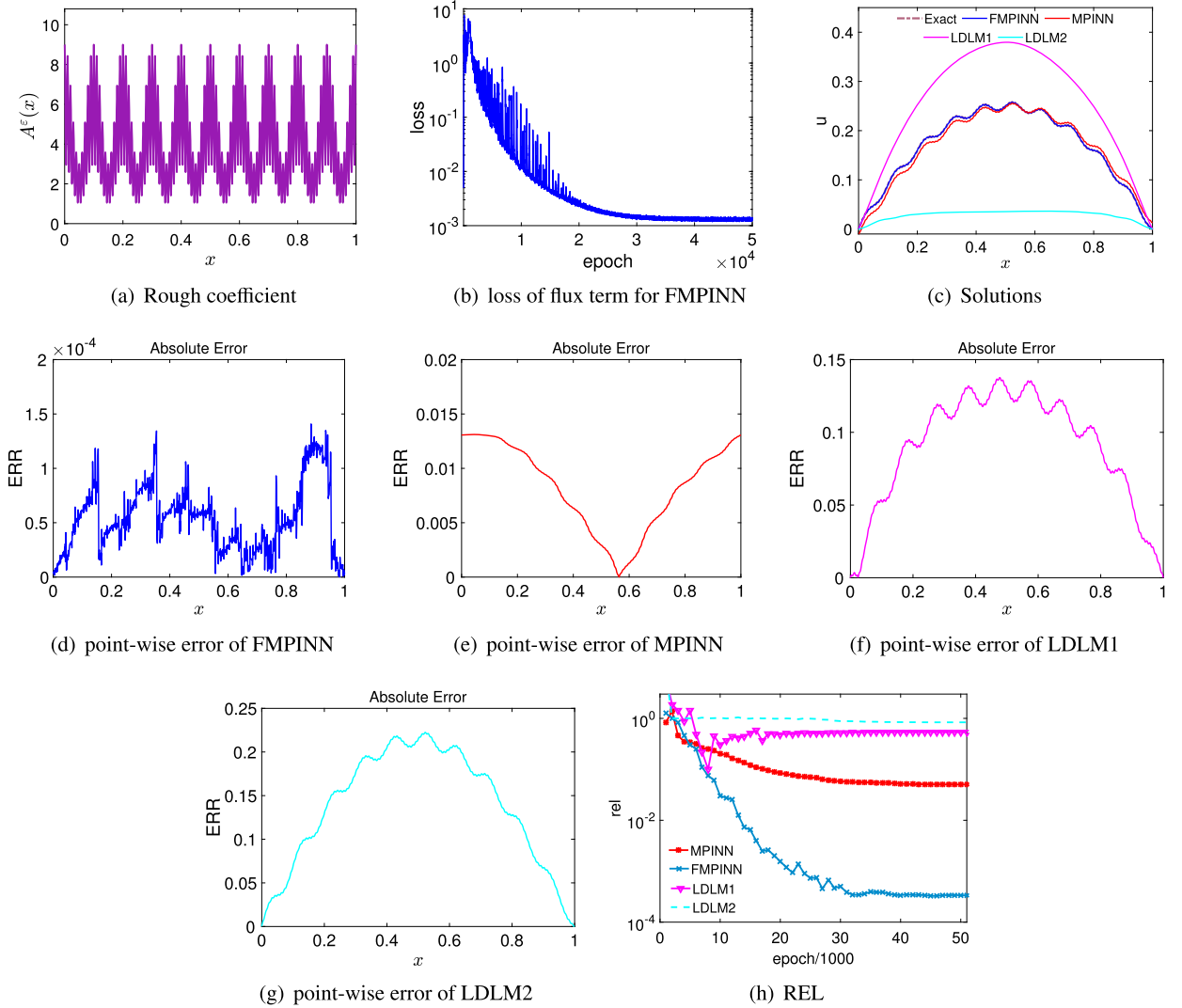


Fig. 10. Rough coefficient, loss of flux term and testing results for Example 2 when $\varepsilon_1 = 0.1$ and $\varepsilon_2 = 0.01$.

Table 8

The relative error and consumed time of FMPINN, MPINN, LDLM1, and LDLM2 for Example 3.

Method	FMPINN	MPINN	LDLM1	LDLM2
REL	0.0139	0.99	0.2431	0.2401
Total time(s)	2098.258	3885.934	626.685	689.619

running time of our FMPINN model is about half of that of the MPINN model, which means the FMPINN model is efficient in solving multi-scale PDEs (1) with two scales coefficient.

Example 4. We consider the following two-dimensional problem for (1) with Dirichlet boundary in regular domains $\Omega = [-1, 1] \times [-1, 1]$. In this example, we choose the $f(x_1, x_2) = 1$ and provide a multi-frequency coefficient

$$A^\varepsilon(x_1, x_2) = \prod_{i=1}^5 \left(1 + 0.5 \cos(2^i \pi(x_1 + x_2)) \right) \left(1 + 0.5 \sin(2^i \pi(x_2 - 3x_1)) \right). \quad (27)$$

Same as the Example 3, a reference solution $u^\varepsilon(x_1, x_2)$ is set as the finite element solution computed by numerical homogenization method [15] on a square grid $[-1, 1] \times [-1, 1]$ with mesh-size $h = 1/128$.

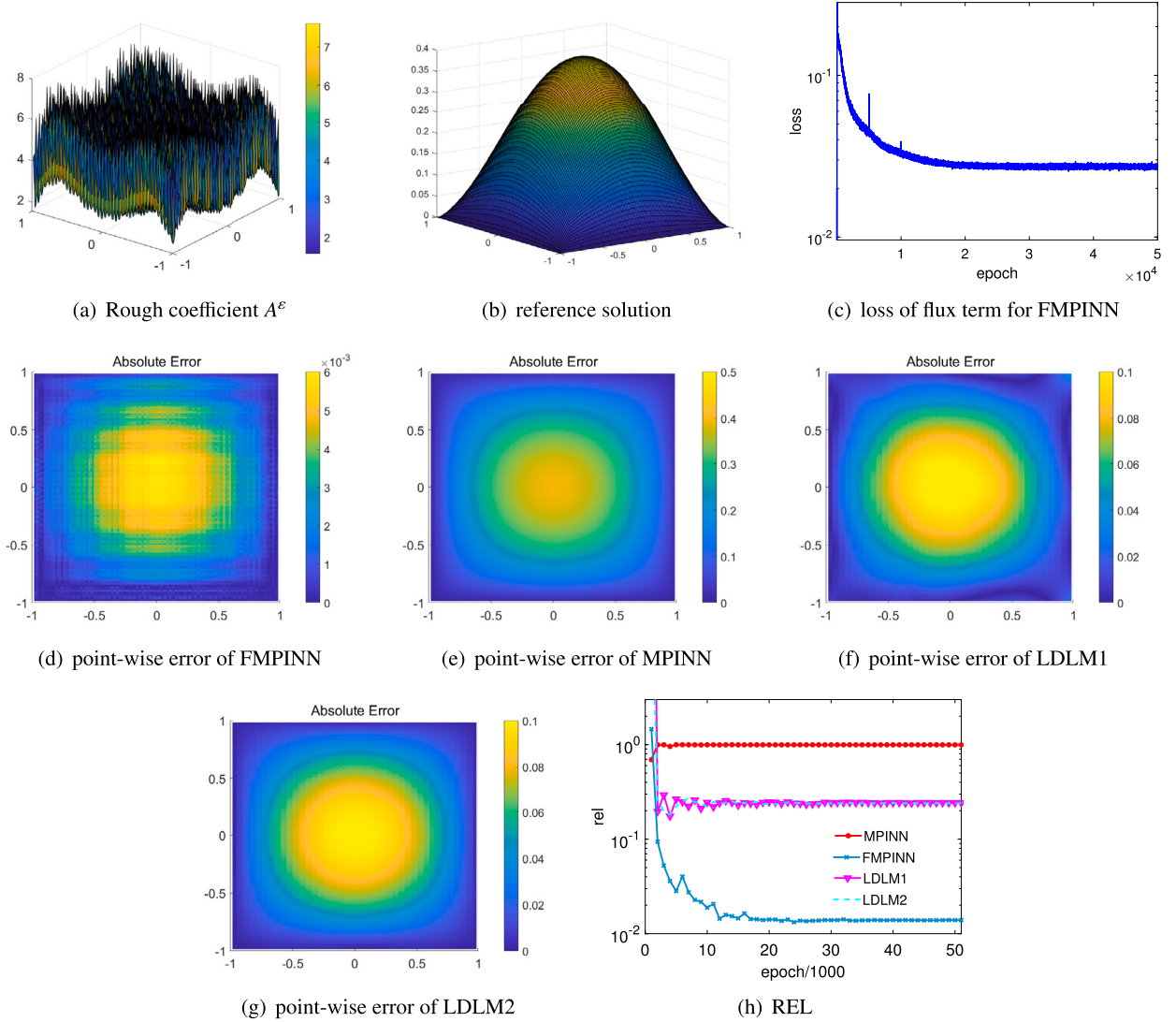


Fig. 11. Rough coefficient, reference solution, loss of flux term and testing results for Example 3.

Table 9

The relative error and consumed time of FMPINN, MPINN, LDLM1, and LDLM2 for Example 4.

Method	FMPINN	MPINN	LDLM1	LDLM2
REL	0.0557	0.99	0.936	0.9127
Total time(s)	2013.258	3985.934	606.685	659.619

By meticulously implementing the previously mentioned FMPINN, MPINN, and LDLMs models with the specified setups, we obtain the approximated solution of (1) with (27). The setup for all models is identical to that of Example 3. During each training step, the training dataset comprises 5000 points randomly sampled from Ω and 2000 boundary points sampled from the boundary $\partial\Omega$, respectively. Meantime, the testing dataset is composed of grid points on the square domain $[-1, 1] \times [-1, 1]$ with mesh size $h = 1/128$. The related experiment results are listed in Table 9 and plotted in Fig. 12, respectively.

In this example, the $A^\epsilon(x_1, x_2)$ is oscillating with six different frequency components (seeing Fig. 12(a)), it will increase the difficulty for DNN to address multi-scale PDEs (1). The point-wise error (Figs. 12(d) - 12(g)) and the relative errors (Fig. 12(h)) indicate that our FMPINN model is still favorable to capture the solution of multi-scale problems with complex multi-frequency coefficient, but the MPINN, LDLM1, and LDLM2 models all performance poorly for approximating the solution of (1). Additionally, the test REL curve in Fig. 12(h) and the curve of loss for flux term in Fig. 12(c) are all flat indicating the FMPINN model is stable

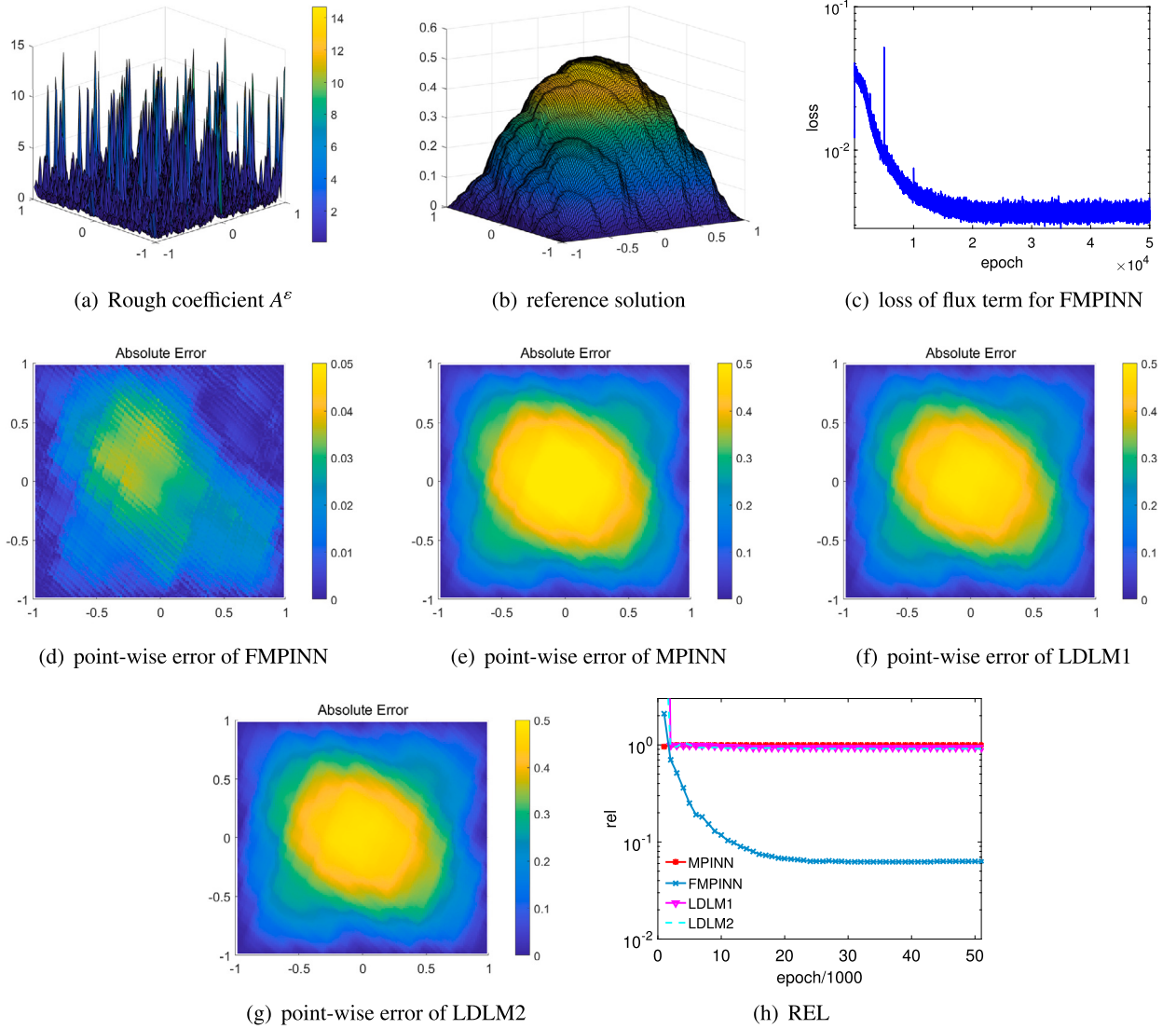


Fig. 12. Rough coefficient, reference solution, loss of flux term and testing results for Example 4.

in the whole training cycle. Moreover, the running time of our FMPINN model is less than that of the MPINN model in solving multi-scale PDEs (1) for coefficient (27).

Example 5. We next study the performance of our FMPINN model to solve the elliptic equation (1) with Dirichlet boundary in a cubic domain $\Omega = [0, 1] \times [0, 1] \times [0, 1]$ where we take

$$A^\varepsilon(x_1, x_2, x_3) = 2 + \sin\left(\frac{2\pi x_1}{\varepsilon}\right) \sin\left(\frac{2\pi x_2}{\varepsilon}\right) \sin\left(\frac{2\pi x_3}{\varepsilon}\right), \quad (28)$$

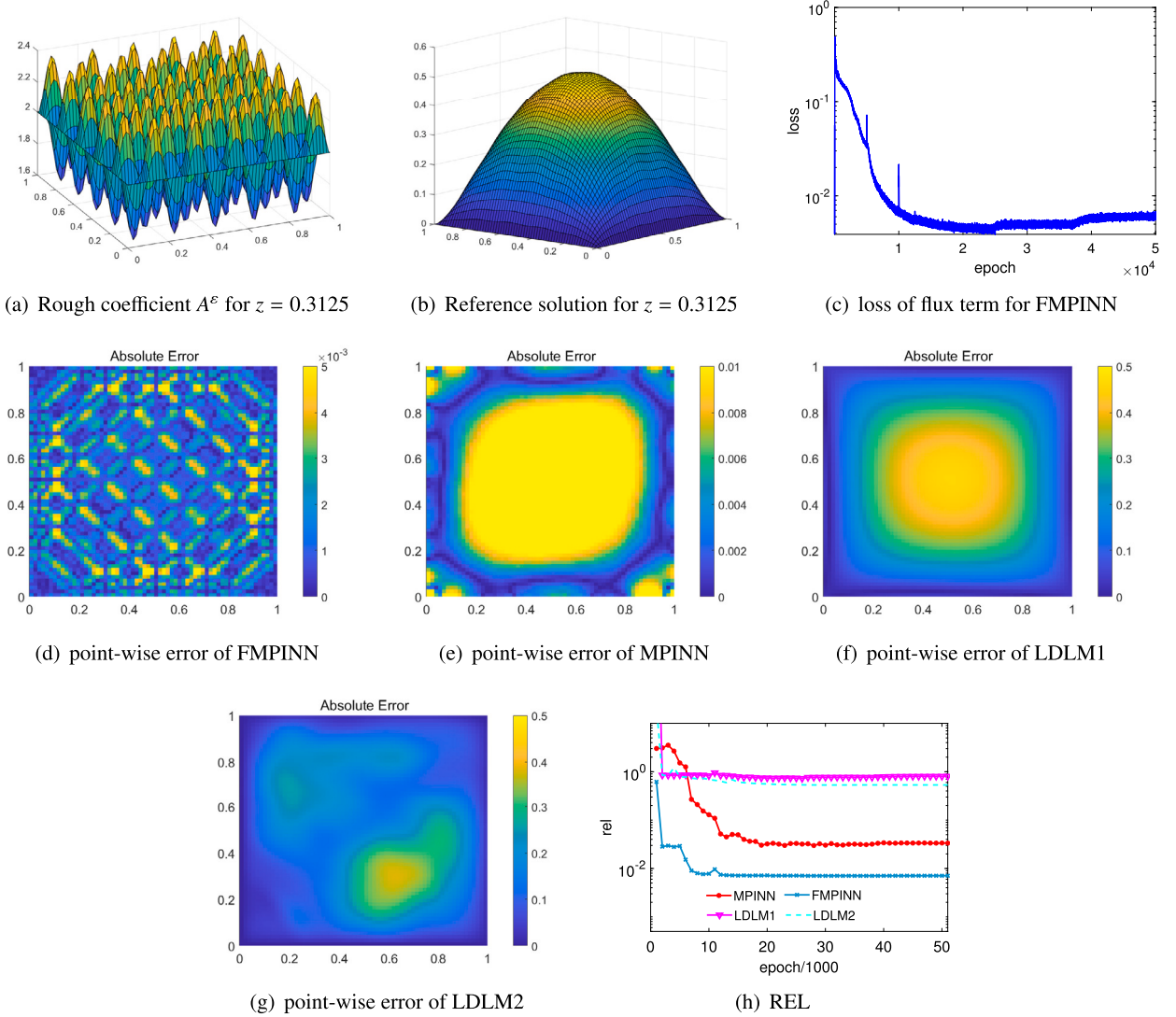
with a small parameter $\varepsilon > 0$ such that $\varepsilon^{-1} \in \mathbb{N}^+$. Also, we let the force side $f(x_1, x_2, x_3) = 20$ and the boundary function $g(x_1, x_2, x_3) = 0$ on $\partial\Omega$.

We utilize the FMPINN, MPINN, LDLM1, and LDLM2 models to approximate the solution of a three-dimensional multi-scale problem (1) with rough coefficient (28) when $\varepsilon = 0.1$, the setups the four models are same as the Example 4. The training dataset includes 7500 interior points and 1000 boundary points randomly sampled from Ω and $\partial\Omega$, respectively. To facilitate the process, a reference solution $u^\varepsilon(x_1, x_2, x_3)$ is established as the numerical solution obtained using the finite difference method on the domain $[0, 1] \times [0, 1] \times [0, 1]$ with a mesh-size $h = 1/64$. The test dataset is formed by including all grid points within the domain $[0, 1] \times [0, 1] \times [0, 1]$ with a mesh size $h = 1/64$ while keeping the value of z fixed at 0.3125. We list the total running time and REL in Table 10 and plot the related results in Fig. 13.

Table 10

The relative error and running time of FMPINN, MPINN, LDLM1 and LDLM2 for Example 5.

Method	FMPINN	MPINN	LDLM1	LDLM2
REL	0.0071	0.0335	0.8048	0.5326
Total time(s)	5179.601	9271.072	1065.541	1195.233

**Fig. 13.** Rough coefficient, reference solution, loss of flux term and testing results for Example 5.

Based on the results in Fig. 13, we can see that our FMPINN model still outperforms the MPINN and LDLMs models for multi-scale problems in three-dimensional space. The point-wise absolute error and the relative error of the former one are much smaller than that of the latter three, the precision of FMPINN is very good with a small absolute point-wise error. Additionally, the REL curve and the loss curve of the flux term are all flat in the later period of the training process, which means the performance of FMPINN is stable. The running time of FMPINN is 5179.601 seconds and less than 3800 seconds than MPINN's.

In addition, we list the H^1 ERR of our proposed FMPINN model for solving Examples 3 – 5 in the following Table 11. The results show that our model can approximate the solution of multi-scale PDEs (1) with satisfactory accuracy.

Example 6. We consider the following eight-dimensional problem for (1) with Dirichlet boundary in regular domains $\Omega = [0, 1]^8$. In which, we take

Table 11
The relative error of FMPINN in H^1 norm for
Examples 3 – 5.

Example	3	4	5
H^1 ERR	0.0394	0.2711	0.0109

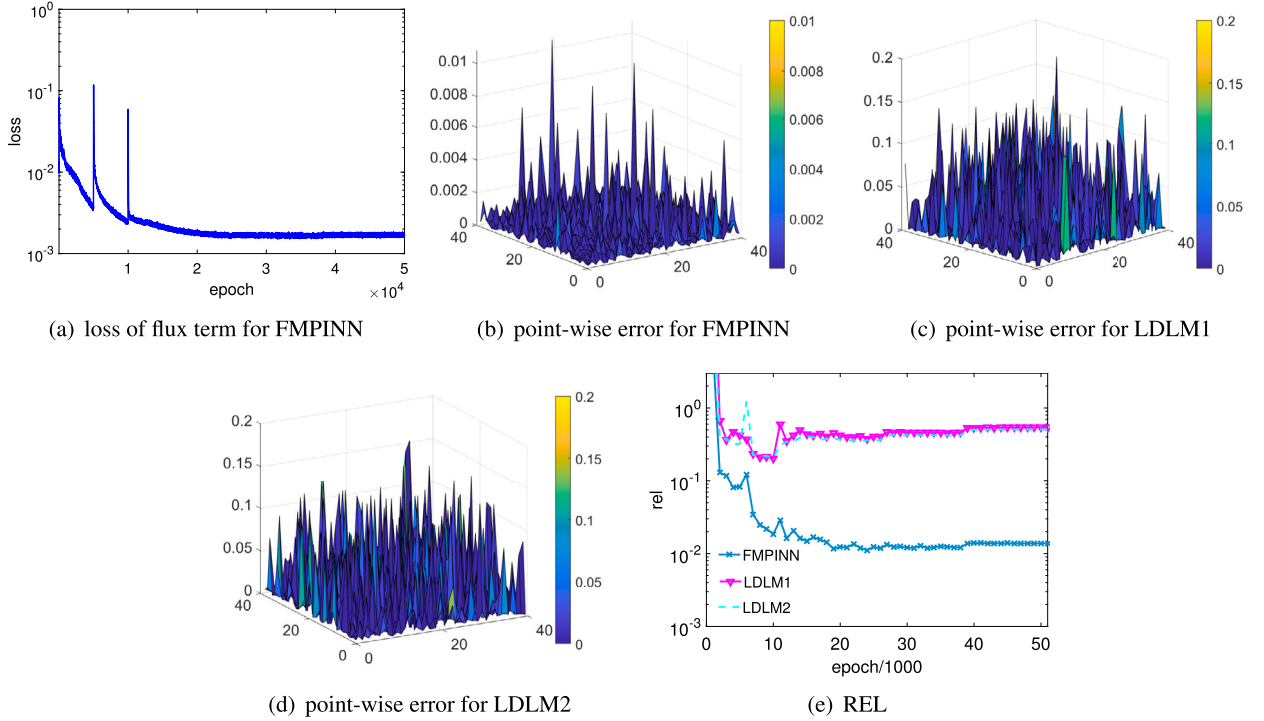


Fig. 14. Loss of flux term and testing results for Example 6.

$$A(x_1, x_2, \dots, x_8) = 1 + \frac{1}{8} \left[\cos(2\pi x_1) + \cos(4\pi x_2) + \cos(8\pi x_3) + \cos(16\pi x_4) + \right. \\ \left. \cos(16\pi x_5) + \cos(8\pi x_6) + \cos(4\pi x_7) + \cos(2\pi x_8) \right].$$

Meantime, an exact solution satisfied (1) is given by

$$u(x_1, x_2, \dots, x_8) = \prod_{j=1}^8 \sin(\pi x_j).$$

The functions $f(x_1, x_2, \dots, x_8)$ in Ω and $g(x_1, x_2, \dots, x_8)$ on $\partial\Omega$ are easy to obtain according to the rough coefficient and exact solution, we omit them.

In this example, we only perform the FMPINN, LDLM1, and LDLM2 model to solve the (1) in eight-dimensional space, because the huge computation requirement of MPINN has exceeded the limitation of memory for our station. The size of hidden layers for each subnetwork of FMPINN is set as (60, 80, 60, 60, 60) and the hidden layers' size for LDLM is set as (400, 500, 300, 300, 300). At each training step, we construct the training dataset by sampling 20000 interior points inside the Ω and 5000 boundary points from the $\partial\Omega$. A testing dataset is given that includes 1600 random points distributed in Ω . The related experiment results are plotted in Fig. 14 and listed in Table 12. Additionally, the point-wise error for the FMPINN model evaluated on 1600 sample points is projected into a rectangular region with mesh size 40×40 . Noting that the mapping is only aimed at visualizing, it is independent of the actual coordinates of those points.

For an eight-dimensional problem, the FMPINN still can obtain a satisfactory solution for (1) with small point-wise absolute error and relative error. However, the LDLM1 and LDLM2 both fail to approximate the solution of (1). Additionally, the loss of flux term and overall REL show that the FMPINN model is also stable during the training process. The running time of LDLMs is less than that of FMPINN in Table 12, but their performance is weaker than that of the latter.

Table 12
The relative error and running time of FMPINN, LDLM1 and LDLM2 for Example 6.

Method	FMPINN	LDLM1	LDLM2
REL	0.01378	0.5394	0.5054
Total time(s)	21035.011	1757.92	1942.742

6. Conclusion

Physics-informed neural networks (PINN) have gained significant popularity in solving both forward and inverse problems. However, the normal PINN with a multi-scale DNN framework is unable to solve multiscale PDEs with rough coefficients. Inspired by the mixed finite element method, this work designs a Fourier-based mixed PINN (dubbed FMPINN) by combining a dual (flux) technique and Fourier decomposition to solve a class of elliptic multi-scale PDEs. By incorporating the loss of the flux term into the loss function, our model achieves favorable stability and robustness. To handle multi-frequency contents, a Fourier activation function has been used to address the input data transformed radially by different frequency factors, and a sub-network is designed to match the target function, this strategy can improve the accuracy and convergence rate for the FMPINN method. Compared to the previous works of PINN, this novel method skillfully casts the original problem into two first-order systems, it will overcome the shortcomings of the computational burden for high-order derivatives in DNN and the ill-condition of neural tangent kernel matrix resulting from the rough coefficient. Computational results show this novel method is feasible and efficient for solving this multi-scale equation with an inhomogeneous coefficient in various dimensional spaces. In the future, we aim to extend this novel network architecture, incorporating Fourier theory and lower-order mixed schemes, to tackle more complex multiscale problems.

CRedit authorship contribution statement

Xi'an Li: Conceptualization, Investigation, Methodology, Validation, Writing – original draft. **Jinran Wu:** Investigation, Writing – review & editing. **Xin Tai:** Writing – review & editing. **Jianhua Xu:** Writing – review & editing. **You-Gan Wang:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

The authors wish to thank Prof. Dr. Zhi-Qin John Xu for valuable suggestions that improved the quality of the paper. In addition, the authors are grateful to the referees for their valuable comments and suggestions which helped to improve the presentation of the paper. This study was supported by the National Key R&D Program of China (No. 2023YFF0717300).

References

- [1] P. Ming, X. Yue, Numerical methods for multiscale elliptic problems, *J. Comput. Phys.* 214 (2006) 421–445.
- [2] E. Weinan, P. Ming, P. Zhang, Analysis of the heterogeneous multiscale method for elliptic homogenization problems, *J. Am. Math. Soc.* 18 (2005) 121–156.
- [3] R. Li, P. Ming, F. Tang, An efficient high order heterogeneous multiscale method for elliptic problems, *Multiscale Model. Simul.* 10 (2012) 259–283.
- [4] A. Abdulle, G. Vilmart, Analysis of the finite element heterogeneous multiscale method for quasilinear elliptic homogenization problems, *Math. Comput.* 83 (2014) 513–536.
- [5] L. Durlafsky, Numerical calculation of equivalent grid block permeability tensors for heterogeneous porous media, *Water Resour. Res.* 27 (1991) 699–708.
- [6] G. Allaire, R. Brizzi, A multiscale finite element method for numerical homogenization, *Multiscale Model. Simul.* 4 (2005) 790–812.
- [7] F. Hellman, A. Målqvist, Numerical homogenization of elliptic pdes with similar coefficients, *Multiscale Model. Simul.* 17 (2019) 650–674.
- [8] T.J.R. Hughes, G. Feijoo, L. Mazzei, J. Quincy, The variational multiscale method—a paradigm for computational mechanics, *Comput. Methods Appl. Mech. Eng.* 166 (1998) 3–24.
- [9] M.G. Larson, A. Målqvist, Adaptive variational multiscale methods based on a posteriori error estimation: Energy norm estimates for elliptic problems, *Comput. Methods Appl. Mech. Eng.* 196 (2007) 2313–2324.
- [10] T. Arbogast, Analysis of a two-scale, locally conservative subgrid upscaling for elliptic problems, *SIAM J. Numer. Anal.* 42 (2004) 576–598 (electronic).
- [11] Y. Efendiev, T. Hou, *Multiscale Finite Element Methods: Theory and Applications*, Springer, 2009.
- [12] Z.M. Chen, T.Y. Hou, A mixed multiscale finite element method for elliptic problems with oscillating coefficients, *Math. Comput.* 72 (2003) 541–576.
- [13] L. Berlyand, H. Owhadi, Flux norm approach to finite dimensional homogenization approximations with non-separated scales and high contrast, *Arch. Ration. Mech. Anal.* 198 (2010) 677–721.
- [14] H. Owhadi, L. Zhang, Homogenization of parabolic equations with a continuum of space and time scales, *SIAM J. Numer. Anal.* 46 (2007) 1–36.

- [15] H. Owghadi, L. Zhang, L. Berlyand, Polyharmonic homogenization, rough polyharmonic splines and sparse super-localization, *ESAIM: Math. Model. Numer. Anal.* 48 (2014) 517–552.
- [16] Y. Efendiev, J. Galvis, T.Y. Hou, Generalized multiscale finite element methods (GMsFEM), *J. Comput. Phys.* 251 (2013) 116–135.
- [17] E.T. Chung, Y. Efendiev, W.T. Leung, An adaptive generalized multiscale discontinuous Galerkin method for high-contrast flow problems, *Multiscale Model. Simul.* 16 (2018) 1227–1257.
- [18] E.T. Chung, Y. Efendiev, W.T. Leung, Residual-driven online generalized multiscale finite element methods, *J. Comput. Phys.* 302 (2015) 176–190.
- [19] A. Målqvist, D. Peterseim, Localization of elliptic multiscale problems, *Math. Comput.* 83 (2014) 2583–2603.
- [20] P. Henning, A. Målqvist, D. Peterseim, A localized orthogonal decomposition method for semi-linear elliptic problems, *ESAIM: Math. Model. Numer. Anal.* 48 (2014) 1331–1349.
- [21] W. E, B. Yu, The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems, *Commun. Math. Stat.* 6 (2018) 1–12.
- [22] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *J. Comput. Phys.* 375 (2018) 1339–1364.
- [23] R.T. Chen, Y. Rubanova, J. Bettencourt, D.K. Duvenaud, Neural ordinary differential equations, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [24] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [25] Y. Khoo, L. Ying, SwitchNet: A neural network model for forward and inverse scattering problems, *SIAM J. Sci. Comput.* 41 (2019) A3182–A3201.
- [26] Y. Zang, G. Bao, X. Ye, H. Zhou, Weak adversarial networks for high-dimensional partial differential equations, *J. Comput. Phys.* 411 (2020) 109409.
- [27] L. Lyu, Z. Zhang, M. Chen, J. Chen, MIM: A deep mixed residual method for solving high-order partial differential equations, *J. Comput. Phys.* 452 (2022) 110930.
- [28] A. Hauptmann, B. Cox, Deep learning in photoacoustic tomography: Current approaches and future directions, *J. Biomed. Opt.* 25 (2020) 112903.
- [29] M. Dissanayake, N. Phan-Thien, Neural-network-based approximations for solving partial differential equations, *Commun. Numer. Methods Eng.* 10 (1994) 195–201.
- [30] A.D. Jagtap, G.E. Karniadakis, Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, in: *AAAI Spring Symposium: MLPS, 2021*, pp. 2002–2041.
- [31] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, *Comput. Methods Appl. Mech. Eng.* 365 (2020) 113028.
- [32] S. Lin, Y. Chen, A two-stage physics-informed neural network method based on conserved quantities and applications in localized wave solutions, *J. Comput. Phys.* 457 (2022) 111053.
- [33] J. Yu, L. Lu, X. Meng, G.E. Karniadakis, Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems, *Comput. Methods Appl. Mech. Eng.* 393 (2022) 114823.
- [34] J. Berg, K. Nyström, A unified deep artificial neural network approach to partial differential equations in complex geometries, *Neurocomputing* 317 (2018) 28–41.
- [35] L. Sun, H. Gao, S. Pan, J.-X. Wang, Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data, *Comput. Methods Appl. Mech. Eng.* 361 (2020) 112732.
- [36] L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, S.G. Johnson, Physics-informed neural networks with hard constraints for inverse design, *SIAM J. Sci. Comput.* 43 (2021) B1105–B1132.
- [37] J. Yang, Q. Zhu, A local deep learning method for solving high order partial differential equations, *Numer. Math., Theory Methods Appl.* (2021).
- [38] Z. Cai, J. Chen, M. Liu, X. Liu, Deep least-squares methods: An unsupervised learning-based numerical method for solving elliptic PDEs, *J. Comput. Phys.* 420 (2020) 109707.
- [39] F.M. Berseteche, J.P. Borthagaray, A deep first-order system least squares method for solving elliptic PDEs, *Comput. Math. Appl.* 129 (2023) 136–150.
- [40] Z.-Q.J. Xu, Y. Zhang, T. Luo, Y. Xiao, Z. Ma, Frequency principle: Fourier analysis sheds light on deep neural networks, *Commun. Comput. Phys.* 28 (2020) 1746–1767.
- [41] N. Rahaman, D. Arpit, A. Baratin, F. Draxler, M. Lin, F.A. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of deep neural networks, in: *International Conference on Machine Learning*, 2019.
- [42] Z. Liu, W. Cai, Z.-Q.J. Xu, Multi-scale deep neural network (MscaleDNN) for solving Poisson-Boltzmann equation in complex domains, *Commun. Comput. Phys.* 28 (2020) 1970–2001.
- [43] B. Wang, W. Zhang, W. Cai, Multi-scale deep neural network (MscaleDNN) methods for oscillatory Stokes flows in complex domains, *Commun. Comput. Phys.* 28 (2020) 2139–2157.
- [44] X.-A. Li, Z.-Q.J. Xu, L. Zhang, A multi-scale DNN algorithm for nonlinear elliptic equations with multiple scales, *Commun. Comput. Phys.* 28 (2020) 1886–1906.
- [45] X.-A. Li, Z.-Q.J. Xu, L. Zhang, Subspace decomposition based DNN algorithm for elliptic type multi-scale PDEs, *J. Comput. Phys.* (2023) 112242.
- [46] S. Wang, H. Wang, P. Perdikaris, On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 384 (2021) 113938.
- [47] A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: Convergence and generalization in neural networks, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [48] A.A. Ramabathiran, P. Ramachandran, SPINN: Sparse, physics-based, and partially interpretable neural networks for PDEs, *J. Comput. Phys.* 445 (2021) 110600.
- [49] S. Li, Y. Xia, Y. Liu, Q. Liao, A deep domain decomposition method based on Fourier features, *J. Comput. Appl. Math.* 423 (2023) 114963.
- [50] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, *Adv. Neural Inf. Process. Syst.* 33 (2020) 7537–7547.
- [51] J. Han, Y. Lee, Hierarchical learning to solve PDEs using physics-informed neural networks, in: *International Conference on Computational Science*, Springer, 2023, pp. 548–562.
- [52] W.T. Leung, G. Lin, Z. Zhang, NH-PINN: Neural homogenization-based physics-informed neural network for multiscale problems, *J. Comput. Phys.* 470 (2022) 111539.
- [53] S.P. Carney, A. Gangal, L. Kim, Physics informed neural networks for elliptic equations with oscillatory differential operators, *arXiv preprint*, arXiv:2212.13531, 2022.
- [54] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [55] D.P. Kingma, J.L. Ba, Adam: A method for stochastic optimization, in: *ICLR 2015: International Conference on Learning Representations 2015*, 2015.
- [56] J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, *J. Mach. Learn. Res.* 12 (2011).
- [57] G. Hinton, N. Srivastava, K. Swersky, Neural networks for machine learning, Lecture 6a: Overview of mini-batch gradient descent, *Dept. Comput. Sci.* 14, 2012, p. 2.
- [58] M. Chen, R. Niu, W. Zheng, Adaptive multi-scale neural network with Resnet blocks for solving partial differential equations, *Nonlinear Dyn.* (2022) 1–20.
- [59] R. Araya, C. Harder, D. Paredes, F. Valentin, Multiscale hybrid-mixed method, *SIAM J. Numer. Anal.* 51 (2013) 3505–3531.
- [60] C.P. Robert, G. Casella, *Monte Carlo Statistical Methods*, 1999.
- [61] K. Gu, P. Fang, Z. Sun, et al., Error analysis of mixed residual methods for elliptic equations, *arXiv preprint*, arXiv:2305.06193, 2023.
- [62] L. Li, X.-C. Tai, J. Yang, Q. Zhu, A priori error estimate of deep mixed residual method for elliptic PDEs, *J. Sci. Comput.* 98 (2024) 44.
- [63] Z. Wang, M. Chen, J. Chen, Solving multiscale elliptic problems by sparse radial basis function neural networks, *J. Comput. Phys.* 492 (2023) 112452.

- [64] J. Chen, X. Chi, Z. Yang, Bridging traditional and machine learning-based algorithms for solving PDEs: The random feature method, *J. Mach. Learn.* 1 (2022) 268–298.
- [65] S. Ding, X. Xu, R. Nie, Extreme learning machine and its applications, *Neural Comput. Appl.* 25 (2014) 549–556.
- [66] F. Calabrò, G. Fabiani, C. Siettos, Extreme learning machine collocation for the numerical solution of elliptic PDEs with sharp gradients, *Comput. Methods Appl. Mech. Eng.* 387 (2021) 114188.
- [67] Z. Xiang, W. Peng, W. Zhou, W. Yao, Hybrid finite difference with the physics-informed neural network for solving PDE in complex geometries, *arXiv preprint, arXiv:2202.07926*, 2022.
- [68] F.A. Viana, A tutorial on Latin hypercube design of experiments, *Qual. Reliab. Eng. Int.* 32 (2016) 1975–1985.
- [69] J. Shaw, A quasirandom approach to integration in Bayesian statistics, *Ann. Stat.* (1988) 895–914.
- [70] M.B. Giles, Multilevel Monte Carlo methods, *Acta Numer.* 24 (2015) 259–328.
- [71] G. Yuan, X. Lu, An active set limited memory BFGS algorithm for bound constrained optimization, *Appl. Math. Model.* 35 (2011) 3561–3573.